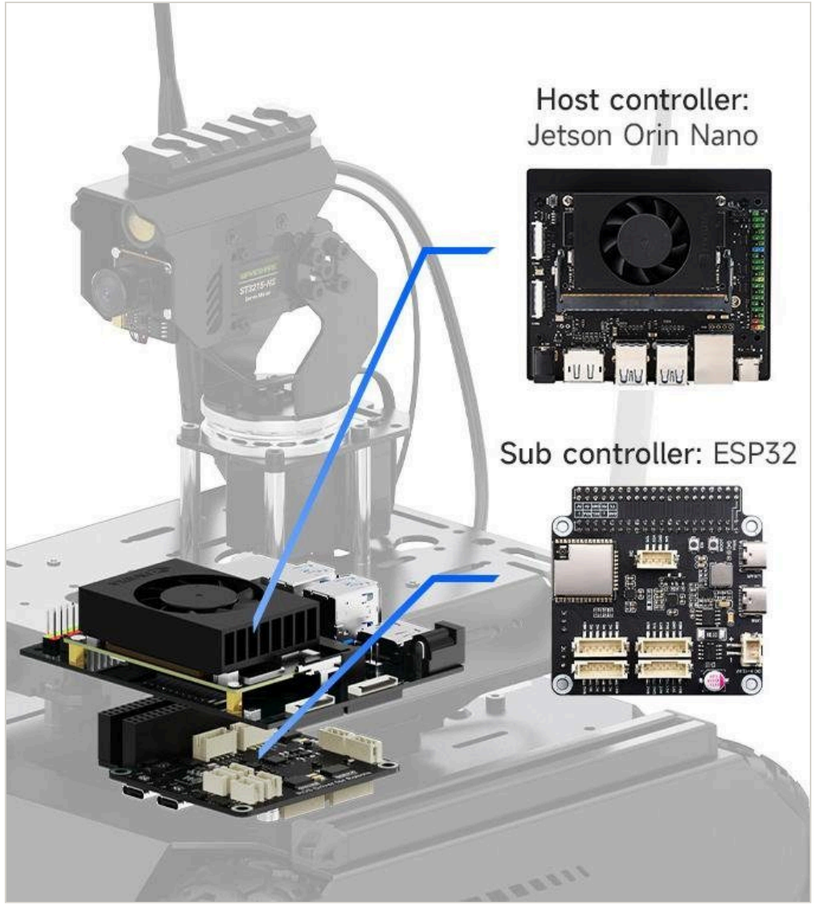
윙봇 하드웨어, 손으로 이해하기

세팅 · 핀 구성 · 장치와 센서의 원리 · 설치 · 고장 났을 때 할 일 — WB-EDU 교육용 로봇 하드웨어 집중 과정

전원은 단계마다 재고,
고장은 전원 → 연결 → 신호 순서로 찾는다.

강의 · 강민호

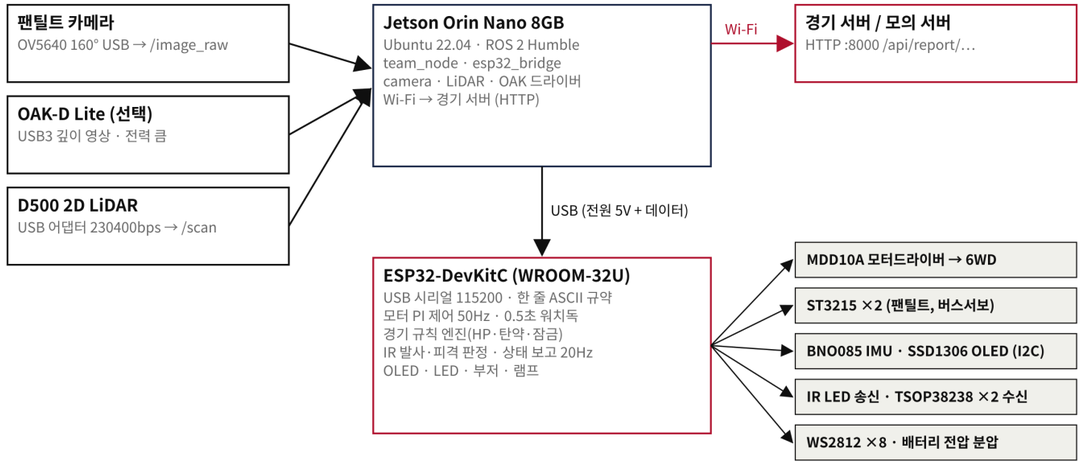


오늘의 길

하드웨어 과정 한눈에

1. **구조와 부품** — 두뇌·손발·감각, 무엇이 들어 있나
2. **작업 준비** — 공구·전선·커넥터·납땜
3. **전기 기본** — 옴의 법칙·PWM·모터 전류·MOSFET·전원·배터리
4. **핀 구성과 결선** — 핀 고르는 원칙, 커넥터별 결선표, 배선 순서
5. **장치·센서** — 하나씩: 원리 → 설치 → 고장 시
6. **설치 확인과 보정** — 부품별 시험, USB 이름, 센서, 실측 보정
7. **고장 대응** — 진단 순서, 멀티미터, 증상표, 교체, 경기 당일 점검

WB-EDU 시스템 구조 — 두뇌(Jetson) · 손발(ESP32) · 감각(센서)



※ 교육용 호환 세트의 구성입니다. 공식 WINGBOT-UGV 내부 구성과 다를 수 있습니다.

이 과정의 약속

① 전원 연결은 **마지막** ② 한 단계 연결 → 한 번 측정 ③ 바퀴는 **띄우고** 시험 ④ 이상하면 **먼저 끈다**

읽는 법

* 표시 읽는 법 — 대회용과 교육용 구분

기능	대회용 — 공식 ROS 2 API (실제 구동)	교육용* — ESP32 펌웨어 제안
주행	`/cmd_vel` 발행 (linear.x · angular.z, 10Hz)	직렬 명령 `V` → MDD10A PWM · 엔코더 PI
팬틸트	`/mil/gimbal` 서비스 (pan · tilt −100~100)	직렬 명령 `G` → ST3215 버스서보 패킷
레이저 발사	`/mil/fire` 서비스 (fired · empty · locked · reloading)	직렬 명령 `F` → 적외선 LED, 38kHz(예시) + 식별코드
피격 판정	로봇·표적의 수광센서가 자동 감지, 서버로 자동 전송	TSOP38238 수신기 + 펌웨어 규칙 엔진
상태 · 경기 단계	`/mil/state` · `/mil/match` 구독	직렬 보고 `T` · 심판 명령 `S`
표시	`/mil/oled` 발행 (4줄 · 21자)	직렬 명령 `O` → SSD1306
카메라 · IMU	`/image_raw` (640×480 BGR) · `/imu/data`	USB 카메라 노드 · BNO085

꼭 구분하세요

대회용 로봇 안의 ESP32 펌웨어 · 핀 배치 · 회로 · 레이저 변조 주파수는 **공개되지 않았고, 참가팀이 고치지도 않습니다** (자료: “로봇 운영 프로그램을 직접 수정할 필요가 없습니다”). 이 자료에서 ***** 가 붙은 장의 핀 번호 · 회로 · 펌웨어 · 38kHz(예시) 는 **원리를 배우기 위한 교육용 제안**입니다. 대회에서는 가운데 칸(대회용)의 API 만 씁니다. 장마다 위쪽에 **대회용 / 교육용** 표시가 있습니다.

PART 1

구조와 부품

로봇은 두뇌(Jetson) · 손발(ESP32) · 감각(센서)으로 나뉜다. 무엇이 어디 붙는지부터 본다.

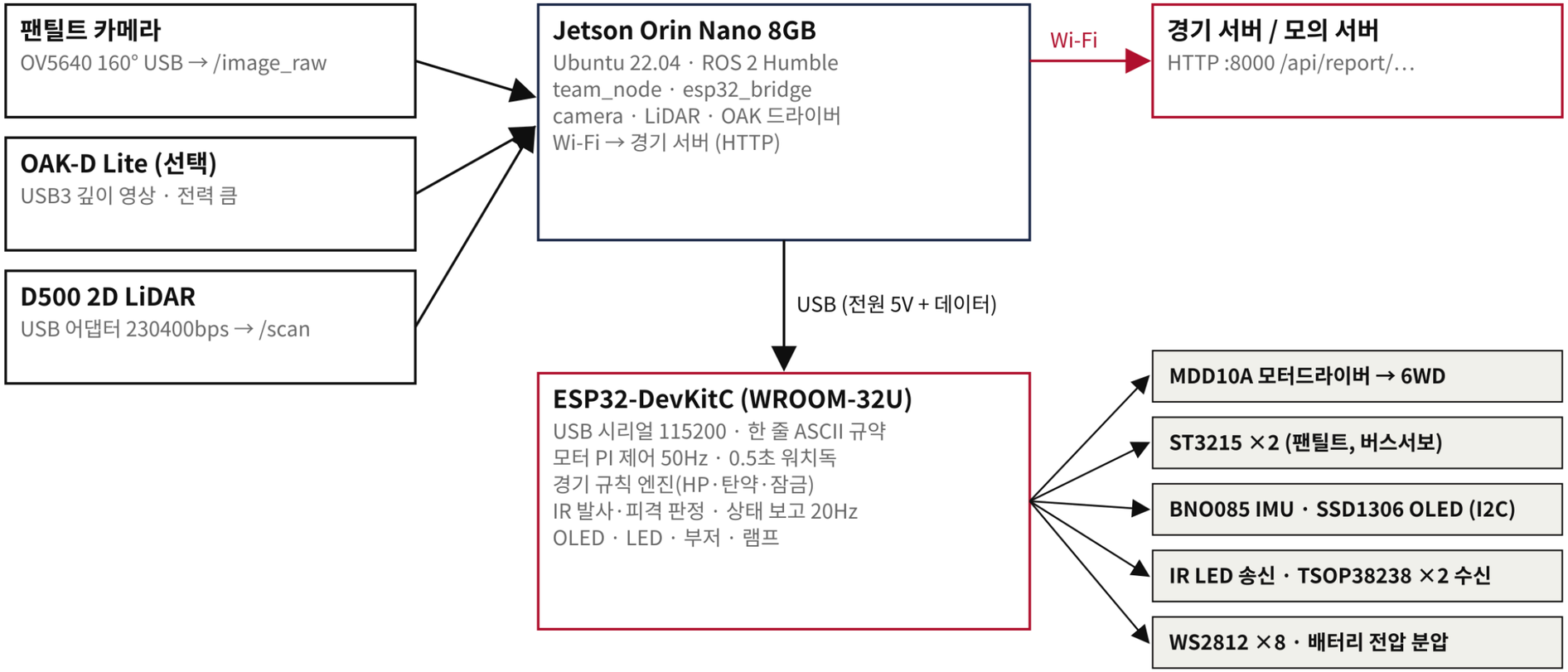
역할을 알면, 고장 난 곳도 좁혀진다.



구조

두뇌 · 손발 · 감각

WB-EDU 시스템 구조 — 두뇌(Jetson) · 손발(ESP32) · 감각(센서)



※ 교육용 호환 세트의 구성입니다. 공식 WINGBOT-UGV 내부 구성과 다를 수 있습니다.

대회용 실물

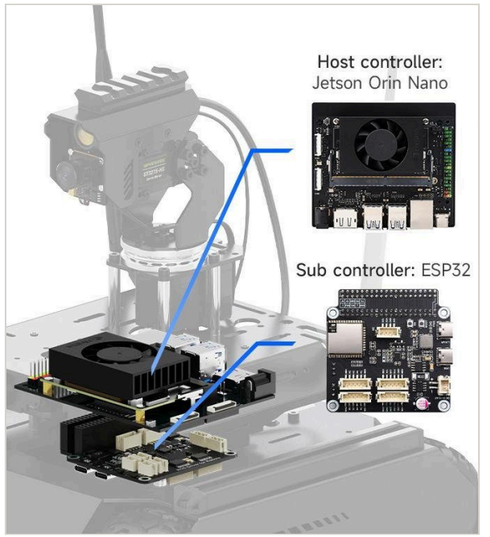
대회 로봇 WINGBOT-UGV — 이렇게 생겼다



앞 · 팬틸트 카메라와 레이저, 빨간 피아식별 안테나



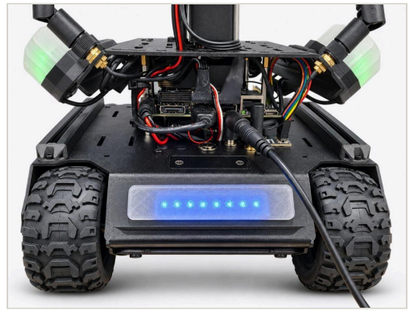
옆 · 6륜, 위로 솟은 2축 팬틸트



속 · 위 Jetson Orin Nano(두뇌), 아래 ESP32(손발)



위에서 · 6WD 배치



뒤 · 탄창 LED 바

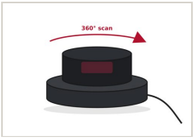
WB-EDU 도 같은 구조

위층 Jetson + 아래층 ESP32, 팬틸트 위 카메라·발사기, 좌우 수광센서, 6륜.

대회용 부품표 ①

대회 자료(CSV)에 있는 장치 — 교육용 선택



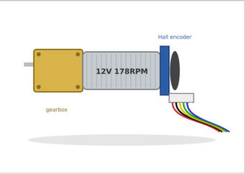


#	역할	교육용 로봇 모델	수량	CSV 참고가	확인할 점
2	AI 컴퓨터	Jetson Orin Nano Super 개발자킷 8GB	1	645,000원	로봇 내장 모듈과 캐리어보드·전원포트 다름
5	팬틸트 카메라	OV5640 5MP 160° USB	1	US$21.60	USB(UVC) 형인지 확인 — MIPI 형은 다른 드라이버
6	깊이 카메라	Luxonis OAK-D Lite	1	539,000원	선택. USB3, 전력 큼
7	2D LiDAR	Waveshare D500 Kit	1	US$95.99	USB 어댑터 포함. 드라이버 호환 확인
9	팬틸트 서보	ST3215 **12V형**	2	US$16.99~21.99	7.4V형과 혼동 금지
10	하위 제어기	ESP32 (그림: 보조 제어기 참조 · 교육용은 DevKitC)	1	8,800원	32U = 외장 안테나형 (안테나 꼭 연결)
13	IMU	Adafruit BNO085	1	US$29.50	대회 IMU 와 동일 모델 아님
21	LED	WS2812B 8구 바	1	—	탄창 8발 표시
22	OLED	SSD1306 0.96" 128×64 I2C	1	8,030원	주소 0x3C
16	충전기	Li-ion 3S 12.6V 2A	1	10,500~27,000원	커넥터 극성 확인

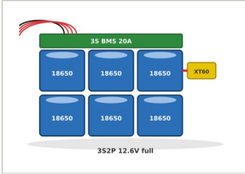
가격은 CSV(2026-11-09 확인) 기준이며 바뀔 수 있습니다. CSV 14번 UPS 모듈(21700 전제)은 이 배터리와 맞지 않아 **쓰지 않습니다**.

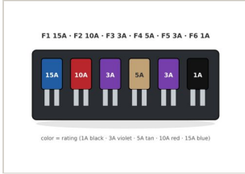
교육용 부품표 ②

CSV 에 없지만 반드시 필요한 부품*







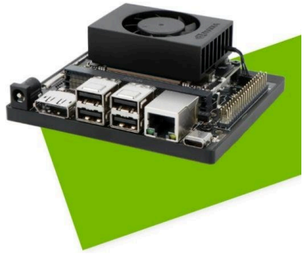


분류	부품	수량	고른 이유
구동	JGB37-520 12V **178RPM** 홀 엔코더 모터 + 65mm 바퀴	6	65mm × 178RPM ≈ 0.6m/s = 대회 최고속도와 같음
구동	Cytron **MDD10A** 2채널 모터 드라이버	1	채널당 10A 연속 · 3.3V 신호 OK · 전압강하 작음
서보	Waveshare Bus Servo Adapter (A)	1	ESP32 UART ↔ ST3215 한 선 버스 변환
전원	3S 18650 팩(보호회로 포함, 30Q×6) 또는 셀 + **3S 20A BMS**	1	셀 직접 납땜 금지 → 완제품 팩 권장
전원	20A 로커 스위치 · 블레이드 퓨즈 홀더 6구 · XT60	1식	갈래마다 퓨즈
전원	승강압 DC-DC 19V 5A / 강압 DC-DC 5V 3A	각 1	Jetson 19V · LED·IR 5V
전원	DC 잭 **5.5/2.1**(충전) · **5.5/2.5**(Jetson)	각 1	모양을 다르게 → 잘못 꽂기 방지
신호	74AHCT125 · AO3400A ×3 · 1N4148 ×2		레벨 변환 · 스위치 · 플라이백
IR	TSAL6200 940nm LED · TSOP38238 ×2 · 빨간 LED		모의 레이저 송신·수신·표시
기타	수동 부저 · 12V 램프 · 저항(39Ω 1/2W, 100Ω, 330Ω, 1k, 10k, 22k, 100k) · 100nF		표시·분압·풀업
시험	멀티미터 · 바퀴 띄움 거치대 · (권장) 전류제한 직류전원장치		CSV 교육추천 항목

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

부품 도감 ①

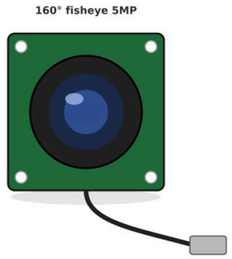
두뇌와 감각 — 생각하고 보는 부품



AI 컴퓨터
Jetson Orin Nano 8GB
ROS 2 · 영상 AI · 경기 서버 통신



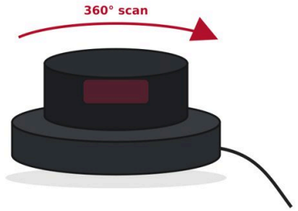
하위 제어기
ESP32 (그림: 보조 제어기 참조 · 교육용은 DevKitC)
모터·서보·IR·LED 정확한 타이밍



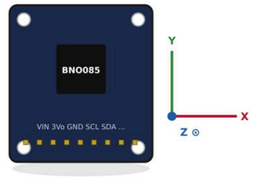
팬틸트 카메라
OV5640 5MP 160° USB
/image_raw 640×480 · 물체 인식



깊이 카메라 (선택)
Luxonis OAK-D Lite
두 렌즈 차이로 거리 계산



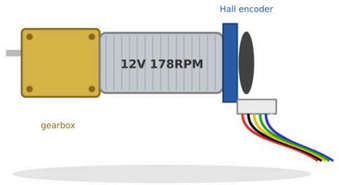
2D LiDAR
Waveshare D500
360° 거리 → 지도·장애물



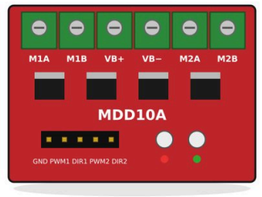
IMU
Adafruit BNO085
기울기·회전·충격

교육용 부품 도감 ②

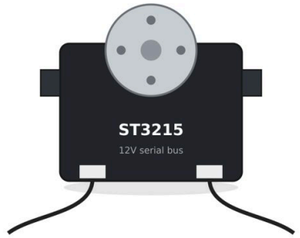
손발과 표시 — 움직이고 알려 주는 부품*



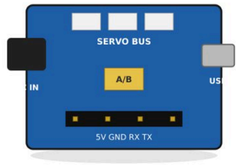
구동 모터 ×6
JGB37-520 12V 178RPM 엔코더
금색 감속기 + 뒤쪽 홀 엔코더



모터 드라이버
Cytron MDD10A
채널당 10A · DIR+PWM



팬틸트 서보 ×2
ST3215 12V 버스서보
선 하나에 ID 로 구분



버스서보 어댑터
Waveshare Bus Servo Adapter (A)
ESP32 UART ↔ 서보 버스



OLED
SSD1306 0.96" 128×64 I2C
4줄 × 21자 상태 표시

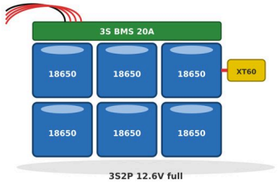


LED 바
WS2812B 8구
탄창 8발 · 피격 색

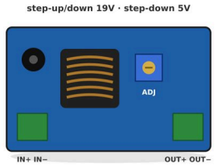
* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 부품 도감 ③

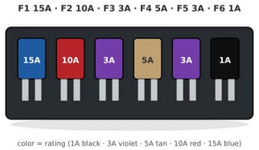
전원과 모의 레이저 — 힘을 주고 빛을 쏘는 부품*



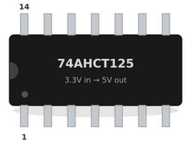
배터리 + BMS
18650 3S2P · 3S 20A BMS
12.6V 만충 · 보호회로



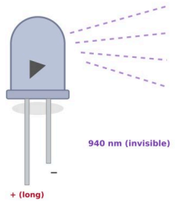
DC-DC 컨버터 ×2
19V 승강압 · 5V 강압
출력은 연결 전에 맞추기



퓨즈 블록
블레이드 퓨즈 6구
색 = 정격 (빨강 10A)



레벨 변환
74AHCT125
3.3V → 5V (WS2812)



IR LED
TSAL6200 940nm
모의 레이저 송신



IR 수신기 ×2
TSOP38238
38kHz(예시) 만 골라 듣기



MOSFET ×3
AO3400A (SOT-23)
IR·램프·부저 스위치

작은 부품 고르는 팁
AO3400A 는 3mm 크기 표면실장 부품 → 납땜이 어렵다면 **SOT-23 변환기판**에 올린 것을 사세요. TSOP 는 다리 순서가 제조사마다 다를 수 있어 데이터시트로 OUT/GND/VS 를 꼭 확인합니다.

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

대회용 **지급 구성품**

상자에 들어 있는 것 — 두 어댑터를 헷갈리지 말 것



보관 케이스



배터리 충전기 12.6V 2A



Jetson 어댑터 — 충전 포트 금지



HDMI to DP 케이블

둘 다 검은 네모 어댑터입니다

충전기와 Jetson 어댑터는 겉모양이 비슷합니다. 받자마자 **라벨 스티커(‘충전용’ / ‘Jetson 전용’)**를 붙이세요. 교육용 로봇는 잭 크기도 다르게(5.5/2.1 충전 · 5.5/2.5 Jetson) 해서 잘못 꽂히지 않게 했습니다.

PART 2

작업 준비

공구 · 전선 · 커넥터 · 납땜. 좋은 결선은 좋은 준비에서 나온다.

부품이 타는 원인의 대부분은 회로가 아니라 작업대에 있다.



작업대

공구와 안전장비 — 이것만은 꼭

공구	왜 필요한가	사용 요령
멀티미터	전압·통전·저항 측정	자동 범위 모델이 편함, 통전 '삐' 기능 필수
전류 제한 직류전원 장치	첫 전원 때 합선이 있어도 부품 보호	12V · 1A 제한으로 시작
인두 + 거치대	납땜	330~350°C, 쓰지 않을 땐 거치대에
와이어 스트리퍼 · 니퍼	피복 벗기기·자르기	심선이 끊기지 않게 굵기 칸 맞추기
압착기(크림퍼)	XT60·JST·듀폰 단자	단자 종류에 맞는 다이 사용
열수축 튜브 · 열풍기	납땜 부위 절연	**납땜 전에** 미리 끼워 두기
핀셋 · 돋보기	작은 핀 확인	핀 번호 인쇄 확인용
정전기 방지 매트·손목띠	ESD 로 칩 손상 방지	보드 만지기 전 금속 접지 만지기
보안경 · 배터리 안전백	납 튐 · 배터리 사고 대비	충전은 안전백 안에서, 자리 비우지 않기

사람 안전

인두 끝은 300°C 이상 — 거치대 밖에 두지 않기 · 납땜 연기는 환기 · 납 만진 손은 씻기 · 배터리는 금속 공구와 같은 서랍에 두지 않기

부품 안전

전원 켠 채 배선 바꾸지 않기 · 보드는 정전기 방지 봉투 위에 · 나사·전선 조각이 보드 아래로 들어가지 않게 (합선 원인)

전선과 커넥터

전류에 맞는 굵기, 극성이 틀릴 수 없는 커넥터

구간	최대 전류(대략)	전선	커넥터
배터리 → 스위치 → 12V 버스	모터 6개 + 전체	**16AWG** 실리콘 선	XT60 (극성 고정)
12V → MDD10A → 모터	채널당 수 A (시동 때 더 큼)	18AWG	단자대 나사 조임
12V → 서보 어댑터	서보 2개 정지·시동	20AWG	DC 잭 / 단자
DC-DC → Jetson 19V	Jetson 최대 부하	18~20AWG	DC 배럴 (규격 확인)
신호선 (엔코더·I2C·IR·PWM)	수 mA	24~26AWG	JST-XH / 듀폰 (고정 필요)

색 약속

빨강 = + · **검정 = GND** · 노랑/흰색 = 신호. 색을 지키면 고장 찾기가 반으로 줄어듭니다.

1. 굵은 전선(숫자 작은 AWG)일수록 큰 전류에 덜 뜨거워진다
2. 모터 선은 **시동·막힘 전류** 기준으로 고른다 (평소 전류 아님)
3. 듀폰 핀은 진동에 빠진다 → 핫글루 · 케이블타이 · 잠금형 커넥터
4. 납땜이 **윤기 있고 매끈한 원뿔**이면 좋음, 회색·울퉁불퉁이면 냉납
5. 납땜 후: 당겨 보기 → 통전 '삐' → 이웃 핀과 합선 없음 확인

주의

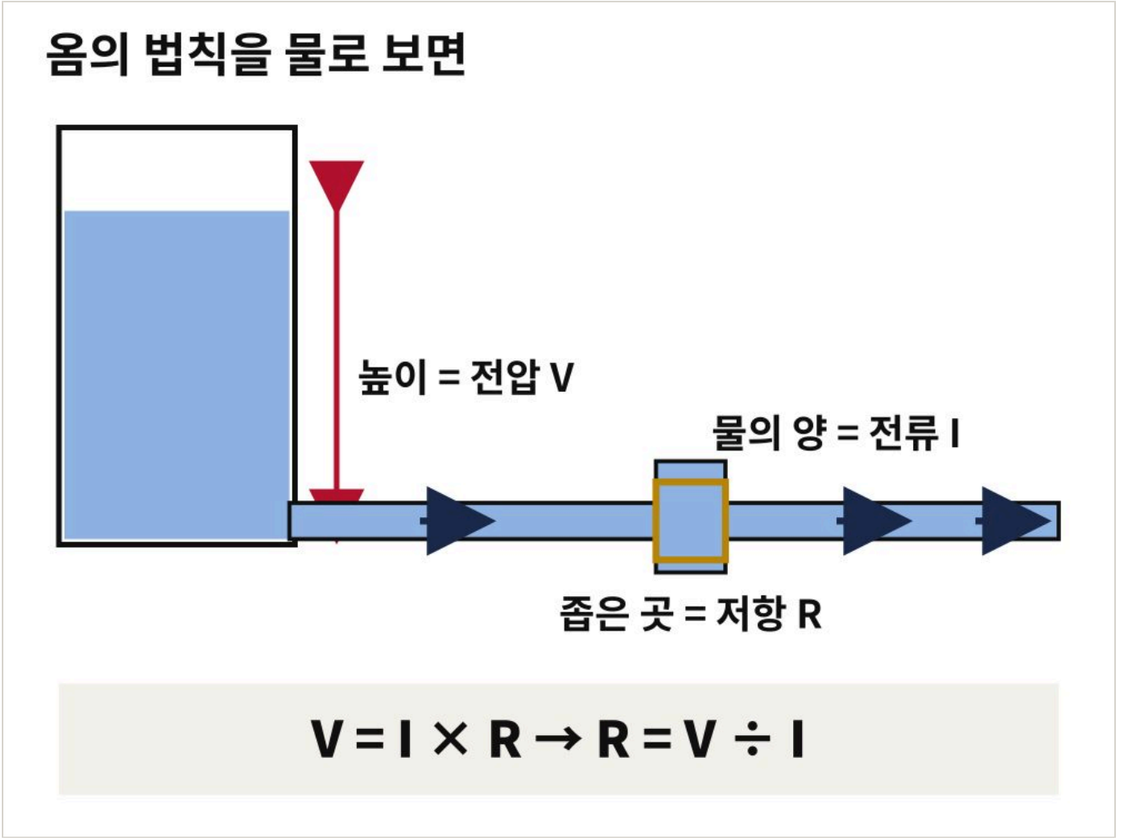
AWG 와 전류 관계는 전선 재질·길이·온도에 따라 다릅니다. 정확한 허용 전류는 **전선 판매처 표**로 확인하세요.

PART 3

전기 기본

옴의 법칙 · PWM · 모터 전류 · MOSFET · 전원 트리 · 배터리.

'켜진다'와 '안전하다'는 다르다.



전압 · 전류 · 저항 — 옴의 법칙 하나로 저항 고르기

전압 V = 전기를 미는 힘 (물의 높이 차)
전류 I = 실제로 흐르는 양 (물의 양)
저항 R = 흐름을 막는 정도 (좁은 관)

$$V = I \times R \;\rightarrow\; R = V \div I$$

예) IR LED 저항

전원 5V, LED 전압강하 약 1.35V, 원하는 전류 0.1A

R = (5 − 1.35) ÷ 0.1 = 36.5Ω → 가까운 **39Ω**

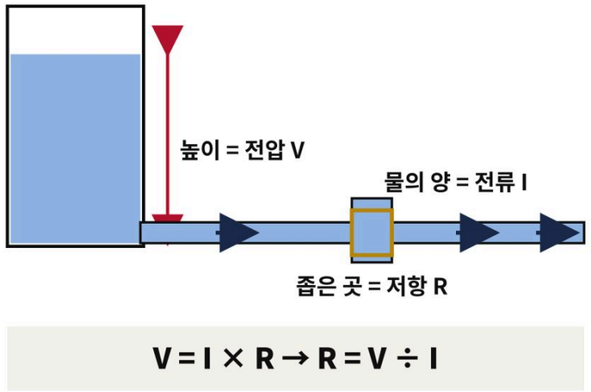
실제 전류 = 3.65 ÷ 39 ≈ **94mA**

전력 P = V × I (저항이 열로 버리는 양)

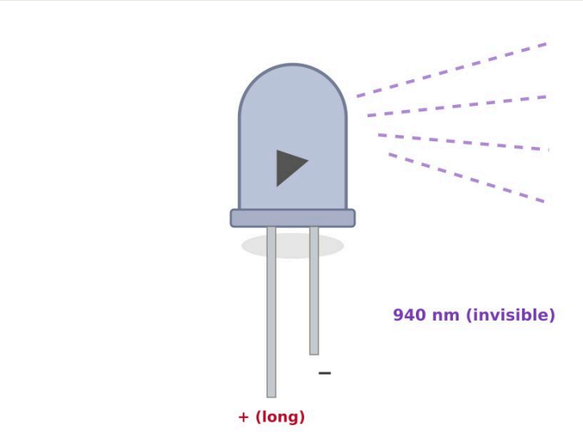
39Ω 저항: 3.65V × 0.094A ≈ 0.34W (켜져 있는 순간)

→ 1/4W(0.25W) 저항은 부족 → **1/2W**

> **팁**
> '불이 켜진다'와 '오래 써도 안전하다'는 다릅니다.
> 1/4W 저항도 처음엔 켜지지만 뜨거워지다 타 버립니다.
> 동작 확인 다음에 꼭 **전력**을 계산하세요.



전압 = 높이, 전류 = 물의 양, 저항 = 좁은 곳



계산 대상: IR LED

전기 기본 ②

PWM 과 GND — 디지털 핀으로 '중간 세기' 만들기

PWM(Pulse Width Modulation, 펄스 폭 변조) 켜짐/꺼짐을 아주 빨리 반복하고, **켜진 비율(duty)**로 평균 세기를 정한다.

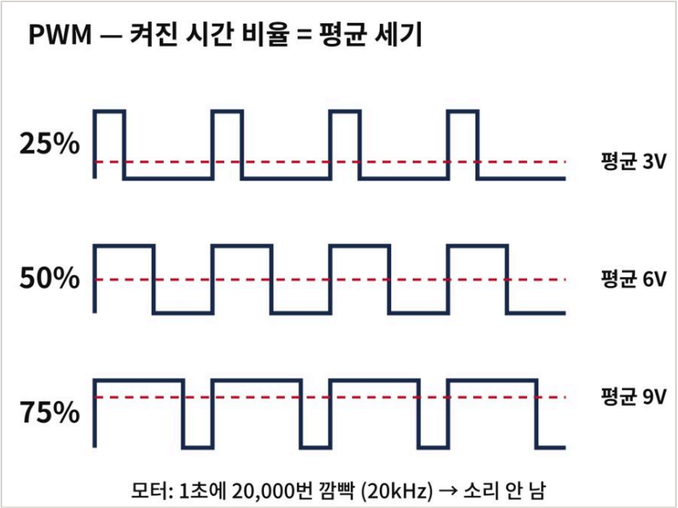
duty	모터에 가는 평균 전압 (12V 기준)
0%	0V
25%	약 3V
50%	약 6V
100%	12V

교육용 로봇: 모터 **20kHz**(소리 안 들림), IR 반송파 **38kHz(예시)**

GND(그라운드) = 모든 전압의 기준점 '0V'

ESP32 가 3.3V 신호를 보낼 때, 받는 쪽(MDD10A)이 그것을 3.3V 로 알아들으려면 **둘의 0V 가 같아야** 한다.

> 공통 GND 원칙
> 모터 전원(12V)을 따로 써도 **GND 는 반드시 연결**.
> GND 가 따로 놀면 신호가 무작위로 HIGH/LOW 로 읽혀 모터가 멋대로 돕니다.



켜진 비율 = 평균 세기

교육용 모터 ①

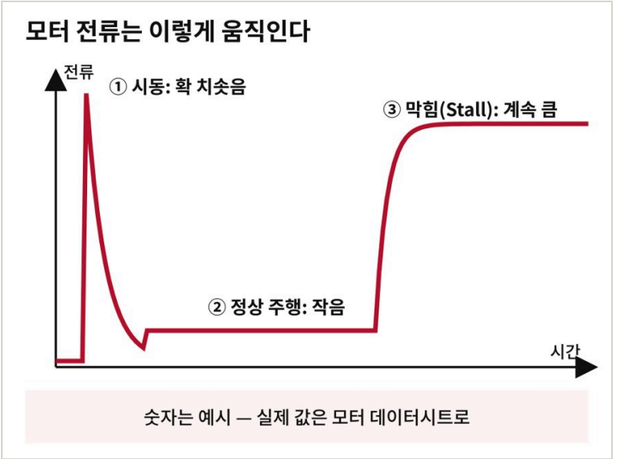
모터는 왜 ESP32 핀에 바로 꽂으면 안 되나*

현상	무슨 일인가	숫자 감각
시동전류 · Stall Current	멈춰 있던 모터가 돌기 시작하거나, 바퀴가 막혀 못 돌 때 전류가 몇 배로 치솟음	정격의 수 배. **판매처 데이터시트로 확인** (모터마다 다름)
역기전력	돌던 모터는 발전기가 됨 → 전원을 끊는 순간 거꾸로 전압이 튐	수십 V 순간 펄스 가능
발열	전류²×저항 만큼 열. 드라이버·배선·모터 모두	드라이버 방열 필수
ESP32 핀 한계	GPIO 1개가 낼 수 있는 전류는 수십 mA, 전압은 3.3V	모터 1개도 못 돌림

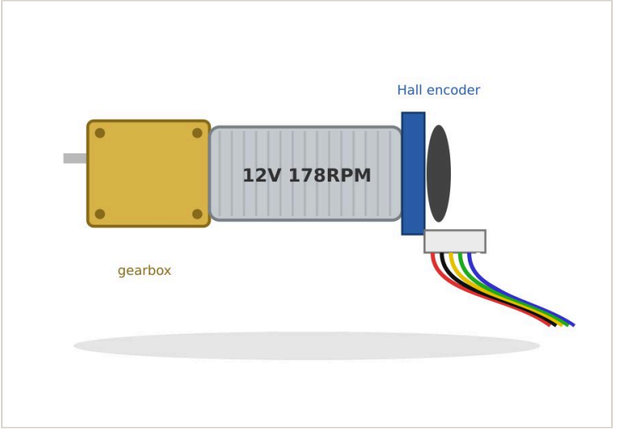
그래서 드라이버(H-브리지)

ESP32 는 '방향(DIR)'과 '세기(PWM)' **신호만** 주고, 큰 전류는 드라이버가 배터리에서 직접 모터로 흘려 줍니다. 역기전력은 드라이버 안의 소자들이 받아 냅니다.

교육용 로봇는 한 채널에 모터 3개 병렬 → 채널 전류 = 모터 1개 × 3. Stall 전류 × 3 이 드라이버 한계(MDD10A 연속 10A) 안인지 꼭 계산.



시동·막힘 때 전류가 치솟는다



JGB37-520 엔코더 모터

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

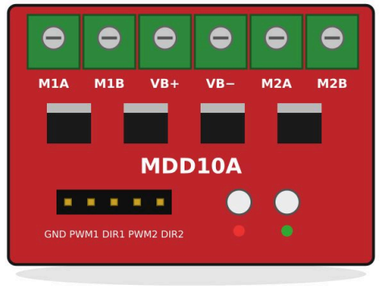
교육용 모터 ②

드라이버 고르기 — L298N 대신 MDD10A 를 쓴 이유*

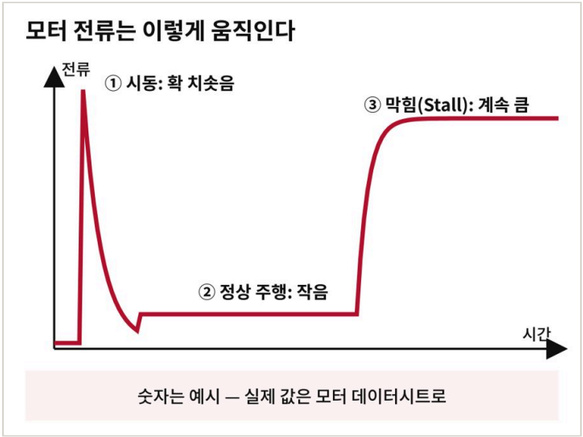
	L298N (흔한 교육용)	Cytron MDD10A (교육용 로봇)
소자	바이폴라 트랜지스터	MOSFET
전압 강하	약 2V 이상 → 12V 가 모터에 약 10V	매우 작음
연속 전류	채널당 약 2A	채널당 10A
발열	큼 (방열판 뜨거움)	작음
입력 신호	IN1/IN2 + **ENA**(PWM)	DIR + PWM (3.3V 직접)
모터 3개 병렬	**부족**	여유

L298N 을 꼭 써야 한다면

보드의 **ENA/ENB 점퍼**가 꽂혀 있으면 PWM 이 무시되고 항상 최고속도입니다 → 점퍼를 빼고 ENA 핀에 PWM 을 연결. 5V 레귤레이터 점퍼는 12V 이하에서만. 방열판이 뜨거우면 이미 한계입니다.



Cytron MDD10A (그림)



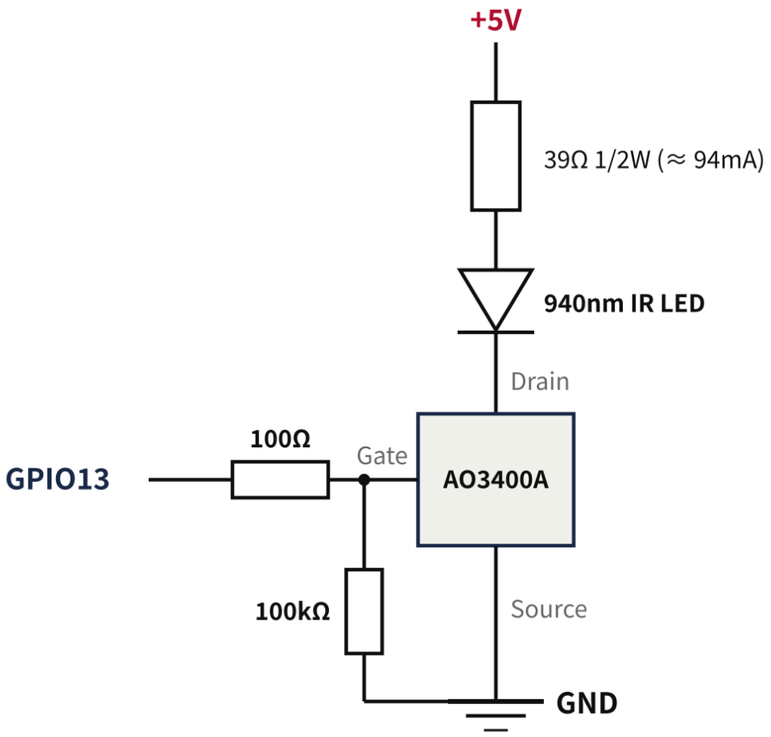
모터 3개 병렬 = 전류 3배

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 MOSFET

작은 신호로 큰 부하 켜기 — 로-사이드 스위치*

MOSFET 로-사이드 스위치 — IR LED · 램프 · 부저 공통 구조



게이트 저항 100Ω

게이트는 작은 콘덴서 → 켜는 순간 돌입전류·울림(링잉) 억제, GPIO 보호

게이트 풀다운 100kΩ

부팅 중 GPIO 가 떠 있을 때(입력 상태) MOSFET 가 저절로 켜지지 않게 0V 로 붙잡음

고르는 법

VGS(th)(켜지기 시작하는 전압) 가 낮다 ≠ 3.3V 에서 잘 켜진다.

데이터시트에서 'VGS = 2.5V 일 때 RDS(on)' 을 확인 (AO3400A: 수십 mΩ 수준)

바디 다이오드

MOSFET 안에 저절로 생긴 다이오드 → 역방향 보호용이 아님

플라이백 다이오드

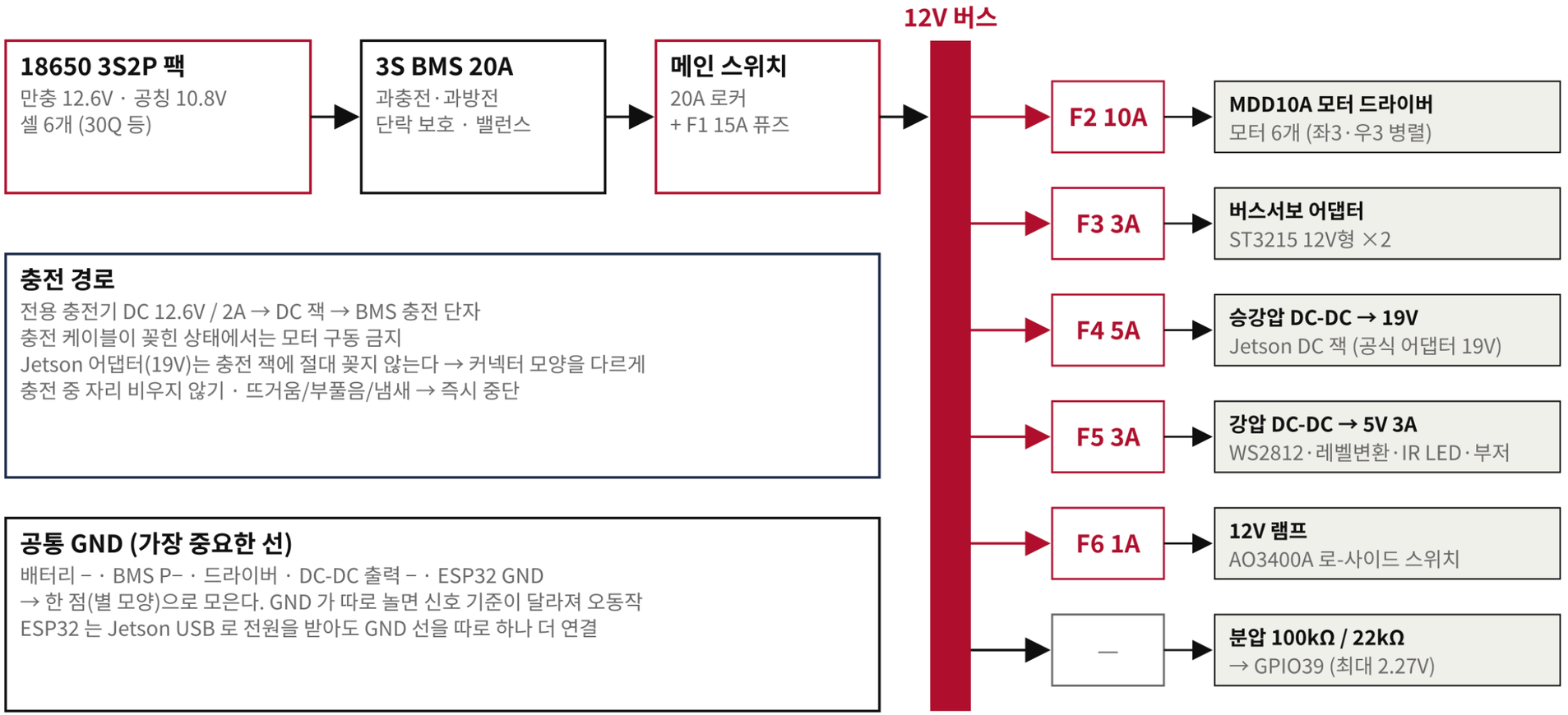
부저·릴레이·모터처럼 코일이 있는 부하는 부하 양단에 1N4148/1N5819 를 거꾸로

→ 끌 때 생기는 역기전력(전압 튐)을 돌려보내 MOSFET 보호. IR LED·램프는 불필요

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

전원 트리 — 갈래마다 퓨즈, GND 는 한 점으로*

전원 트리 — 배터리 하나에서 6갈래, 갈래마다 퓨즈

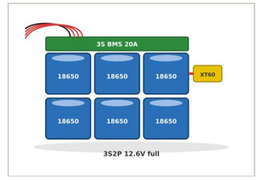


굵기 권장: 배터리~버스 14AWG · 모터 18AWG · 서보/DC-DC 18~20AWG · 신호 24~26AWG

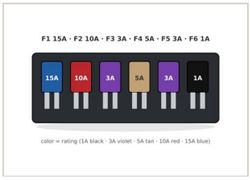
* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 배터리

리튬이온 — '동작한다'와 '안전하다'를 구분하자*







단순 동작	안전한 설계
셀 6개를 묶으면 12V 가 나온다	**BMS** 로 과충전·과방전·단락 보호 + 셀 균형
아무 12V 충전기로 충전	**CC/CV 리튬 전용 12.6V** 충전기만
굵은 선 하나로 다 연결	갈래마다 **퓨즈**, 배터리 + 바로 뒤에 메인 퓨즈
9V 까지 다 쓴다	3S 기준 **9.9V(셀당 3.3V)**에서 멈춤

배터리 사용 규칙

1. 충전 중 자리 비우지 않기, 불연성 바닥 위에서
2. 충전 케이블 꽂힌 채 모터 구동 금지
3. 떨어뜨렸거나 부딪힌 팩: 뜨거움·부풀음·냄새 → 즉시 사용 중지
4. 보관은 50~60% 충전 상태로
5. OLED·`/mil/state` 로 배터리 % 상시 확인 (20% 이하 충전)

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

PART 4

핀 구성과 결선

어떤 핀을 왜 골랐나, 커넥터마다 어디에 꽂나, 어떤 순서로 연결하나.

핀은 아무 데나 꽂지 않는다 — 부팅 핀과 입력 전용 핀을 피한다.



교육용 핀 고르는 원칙

ESP32 핀은 성격이 다르다 — 피해야 할 핀*

핀 종류	GPIO	왜 조심하나	교육용 로봇에서
플래시 전용	6 ~ 11	내부 메모리 연결 — 쓰면 바로 멈춤	**사용 안 함**
스트래핑(부팅 결정)	0 · 2 · 5 · 12 · 15	켜는 순간 값으로 부팅 방식 결정	5 = TSOP(대기 HIGH 라 무해), 15 = 부저(풀다운)
12번 특히	12	켤 때 HIGH 면 플래시 전압이 틀어져 부팅 실패	**사용 안 함**
입력 전용	34 · 35 · 36 · 39	출력 불가, **내부 풀업 없음**	34·35 엔코더(10kΩ 외부 풀업), 39 배터리 ADC
부팅 때 신호가 나오는 핀	1 · 3 · 14 등	켜는 순간 펄스·로그가 나옴	14 는 1kΩ 직렬로 충돌 방지
UART0	1 (TX) · 3 (RX)	USB 업로드·Serial Monitor 용	**사용 안 함**
ADC2	0·2·4·12~15·25~27	Wi-Fi 켜면 ADC2 못 씀	배터리는 ADC1(39)로



참조: 대회용 계열 ESP32 보조 제어기 보드 · 교육용 핀 이름은 DevKitC 보드 인쇄로 확인

5V 금지

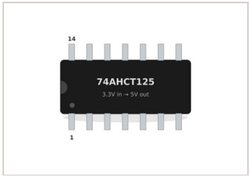
ESP32 GPIO 는 **3.3V**. 엔코더·센서 VCC 를 5V 로 주면 신호도 5V 가 되어 칩이 손상될 수 있습니다. 5V 가 필요한 장치(WS2812)는 **레벨 변환**을 거칩니다.

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 핀 맵

ESP32 핀 배정표 — 모든 GPIO 는 3.3V*





GPIO	연결	방향	주의
25 / 26	MDD10A PWM1 / DIR1 (왼쪽)	출력	PWM 20kHz 10bit
27 / 33	MDD10A PWM2 / DIR2 (오른쪽)	출력	14번은 부팅 때 PWM 이 나와서 피함
34 / 35	왼쪽 엔코더 A / B	입력	**입력전용·풀업 없음** → 10kΩ 를 3.3V 로
18 / 19	오른쪽 엔코더 A / B	입력	엔코더 VCC 는 3.3V 로 (신호도 3.3V)
17 / 16	UART2 TX / RX ↔ 버스서보 어댑터	통신	TX↔RX 엇갈려 연결, 1Mbps
21 / 22	I2C SDA / SCL (OLED 0x3C, IMU 0x4A)	통신	400kHz
4	WS2812 데이터 → 74AHCT125 → 330Ω → LED	출력	3.3V→5V 레벨 변환
13	IR LED MOSFET 게이트	출력	38kHz(예시) 반송파
23	발사 표시 빨간 LED (330Ω)	출력	
14 / 5	TSOP38238 왼쪽 / 오른쪽 (1kΩ 직렬)	입력	둘 다 부팅에 영향 없게 설계
32 / 15	램프 / 부저 MOSFET 게이트	출력	15 는 풀다운 → 부팅 로그만 꺼짐(무해)
39	배터리 분압 (100k/22k)	ADC	인터럽트 금지 핀 — ADC 로만

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 배선도

ESP32 기능별 연결 한눈에*

ESP32-DevKitC 배선도 — 모든 GPIO 는 3.3V



※ 기능별로 묶은 논리 배치도입니다. 실제 보드의 핀 순서와 다르니 납땜·결선은 보드 실크 인쇄(글자)를 보고 하세요.

피한 핀: 6~11(플래시) · 0/2/12(부팅) · 36/39 는 인터럽트 금지(39는 ADC로만 사용) · 34~39 입력전용(풀업 없음)

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 커넥터별 결선표 ①

구동 · 서보 · 전원 측정 — 선 하나하나*

장치 (커넥터)	장치 쪽 핀	→ 연결	선색 약속	확인
MDD10A 신호 헤더	PWM1 · DIR1 · PWM2 · DIR2 · GND	GPIO 25 · 26 · 27 · 33 · ESP32 GND	노랑·흰·노랑·흰·검정	GND 빠지면 모터가 제멋대로
MDD10A 전원	VB+ · VB−	F2 퓨즈 뒤 12V · 공통 GND	빨강 · 검정	역극성 주의
MDD10A 모터 단자	M1A · M1B / M2A · M2B	왼쪽 모터 3개 병렬 / 오른쪽 3개 병렬	모터 선색 그대로	방향은 펌웨어 INVERT 로
엔코더 (모터 6핀)	M+ · M− · GND · VCC · A · B	M± 는 드라이버, GND, **3.3V**, A·B → 34·35 (왼) / 18·19 (오)	모터 케이블 색 기록	34·35 에 10kΩ 풀업
버스서보 어댑터	TX · RX · GND · 전원 입력	GPIO 16(RX) · 17(TX) **교차** · GND · 12V(F3)	—	어댑터 모드 점퍼(UART)
ST3215 데이지체인	3핀 (GND · VCC · 데이터)	어댑터 → 팬 서보 → 틸트 서보	케이블 그대로	ID 1 · 2 로 미리 설정
배터리 분압	100kΩ(위) · 22kΩ(아래) · 100nF	12V 버스 → 분압점 → GPIO 39 · GND	—	분압점 ≤ 2.3V

팁

커넥터를 꽂기 전에 **장치 쪽 인쇄**를 사진으로 찍어 두고, 결선표 옆에 붙이세요. 판매처마다 핀 순서가 다를 수 있습니다.

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 커넥터별 결선표 ②

센서 · 표시 · 모의 레이저 — 선 하나하나*

장치 (커넥터)	장치 쪽 핀	→ 연결	부품	확인
OLED SSD1306	GND · VCC · SCL · SDA	GND · 3.3V · GPIO 22 · GPIO 21	—	주소 0x3C (일부 0x3D)
IMU BNO085	VIN · GND · SCL · SDA	3.3V · GND · GPIO 22 · GPIO 21	—	주소 0x4A, I2C 모드
WS2812 LED 바	5V · DIN · GND	5V · GPIO 4 → 74AHCT125 → 330Ω → DIN · GND	1000µF (5V–GND)	DIN 방향 화살표
IR 발사 TSAL6200	애노드(긴 다리) · 캐소드	5V → 저항 → LED → AO3400A 드레인 · 소스 → GND	게이트 100Ω, 풀다운 100kΩ	GPIO 13 = 게이트
발사 표시 LED	애노드 · 캐소드	GPIO 23 → 330Ω → LED → GND	—	빨간 LED
TSOP38238 ×2	1 OUT · 2 GND · 3 VS	OUT → 1kΩ → GPIO 14(왼) / 5(오) · GND · 3.3V	VS 에 100Ω + 4.7µF	볼록면이 앞
램프 · 부저	+ · –	12V(또는 5V) → 부하 → AO3400A → GND	게이트 GPIO 32 / 15	부저 극성

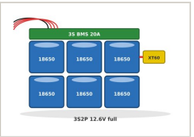
I2C 는 버스

OLED 와 IMU 는 **같은 두 선(SDA·SCL)**을 나눠 씁니다. 주소가 달라서 구분됩니다. 하나가 버스를 망가뜨리면 둘 다 안 보이니, 고장 때는 **하나씩 빼고** 시험하세요.

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 **배선 순서**

전원 없이 연결하고, 한 단계마다 재고 넘어간다*





단계	할 일	멀티미터 확인 (기대값)
1	배터리 팩 → BMS → 메인 스위치 → F1 15A → 12V 버스 (스위치 OFF)	스위치 ON 시 버스 10~12.6V
2	DC-DC 2개 연결 (부하 없이), 출력 조정	19V 모듈 19.0V · 5V 모듈 5.0~5.1V
3	공통 GND 별 모양 연결	각 GND 사이 저항 ≈ 0Ω
4	MDD10A ← F2, 모터 좌3·우3 병렬	모터 단자 사이 단락 없음
5	ESP32 를 Jetson(또는 PC) USB 에만 연결 → 펌웨어 업로드	Serial Monitor 에 `E boot WB-EDU`
6	엔코더·I2C·LED·IR·서보 어댑터 신호선 연결	3V3 핀 3.2~3.4V, 5V 핀 4.8~5.2V
7	분압 저항 연결 → GPIO39	분압점 = 배터리전압 × 0.18 (12.6V → 2.27V)
8	Jetson ← F4 19V (마지막!)	Jetson 전원 LED, 부팅

팁

DC-DC 컨버터의 출력은 **Jetson 에 연결하기 전에** 맞춥니다. 출하 상태 그대로 꽂아서 Jetson 을 태우는 사고가 가장 흔합니다.

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 첫 전원

처음 켤 때 — 바퀴는 띄우고, 손은 스위치에*



1. 로봇을 **거치대**에 올려 바퀴가 바닥에 닿지 않게
2. (권장) 배터리 대신 **전류 제한 직류전원장치** 12V · 1A 제한으로 먼저
3. 메인 스위치 ON → 5초 관찰: 연기·냄새·뜨거운 부품 없음
4. ESP32 부팅음(삐 1번) · OLED 기본화면 · LED 8개 파란색
5. Serial Monitor 에 `E boot ... imu=ok oled=ok`
6. `P` 입력 → `P ok`

OLED 기본화면	뜻
WB-EDU ID11 [P-MODE]	로봇 ID · 연습 모드
HP 4/4 AMMO 8	체력 · 탄약
BAT 87% 12.2V	배터리
DRV AIM	주행·조준 가능

> **MISSING 이 뜨면**
> `imu=MISSING` / `oled=MISSING` → I2C 선(SDA·SCL 바뀜?) · 전원 3.3V · 주소 확인. 펌웨어는 장치가 없어도 나머지 기능으로 계속 동작합니다.

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

JETSON ②

Jetson 40핀 — 3.3V · 번호 두 가지 · ADC 없음



물리 핀 (BOARD)	기능
1 / 17	3.3V
2 / 4	5V
6 / 9 …	GND
3 / 5	I2C SDA / SCL
8 / 10	UART TX / RX
7, 11, 12 …	GPIO

GPIO 는 3.3V. 5V 신호를 넣으면 SoC 가 손상될 수 있음 (ESP32 와 같은 원칙)

번호가 두 가지

`GPIO.setmode(GPIO.BOARD)` → 헤더의 **물리 핀 순서** (1~40)

`GPIO.setmode(GPIO.BCM)` → **칩(BCM) 번호**

예) BOARD 12 = BCM 18 → 코드 맨 위에서 어느 쪽인지 반드시 확인

> **ADC 가 없다**
> Jetson 헤더에는 아날로그 입력이 없습니다. 배터리 전압을 Jetson 이 직접 재려면 ADS1115 같은 외부 ADC 가 필요합니다 → 비교 예제

PART 5

장치와 센서

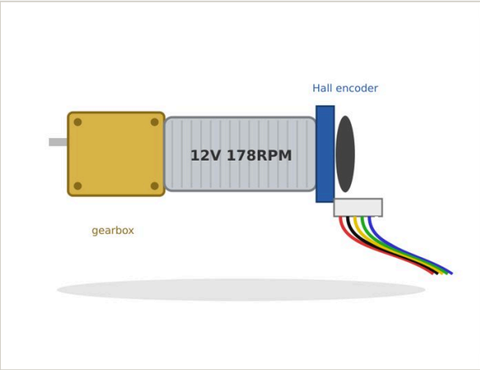
하나씩: 어떤 원리로 움직이나 → 어떻게 다나 → 고장 나면 무엇을 보나.

원리를 알면 고장 증상이 '이유'로 보인다.



교육용 장치 ① 구동

엔코더 모터 JGB37-520 — 바퀴 회전을 센다*



항목	값
전압	12V (DC 기어모터)
엔코더	홀 센서 A·B 2상
펄스	모터축 11 PPR × 감속비
교육용 로봇	가운데 바퀴 좌·우

원리

- 모터축에 붙은 **자석 원판**이 돌면 홀 센서 2개가 번갈아 신호를 낸다
- A·B 는 1/4 주기 어긋남 → **어느 쪽이 먼저** 바뀌는지로 방향을 안다
- 카운트 ÷ (한 바퀴 카운트) × 바퀴 둘레 = 이동 거리
- 기어(감속비)를 거쳐 힘은 커지고 속도는 줄어든다

설치

- 엔코더 VCC 는 **3.3V** (신호도 3.3V 가 됨)
- 34·35 번은 입력 전용 → A·B 에 **10kΩ 풀업**
- 모터 선과 신호선은 **떨어뜨려** 묶기 (잡음)
- 감속비는 라벨마다 다름 → **10바퀴 실측**해 config 수정

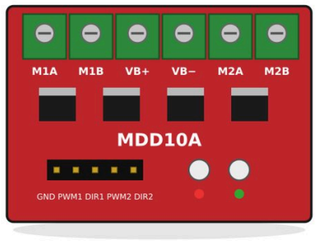
고장 시

- 카운트 0 → 엔코더 전원·GND, A/B 핀 번호
- 카운트가 튐 → 풀업 없음 · 모터선과 같이 묶음
- 앞으로 가는데 − → `ENC_x_INVERT`
- 한쪽 바퀴만 느림 → 기어 걸림·축 고정나사 확인

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 장치 ② 구동

모터 드라이버 MDD10A — 작은 신호로 큰 전류를*



항목	값
채널	2 (좌·우)
입력	PWM + DIR (3.3V 신호 가능)
PWM	최대 20kHz
보호	**퓨즈는 바깥에** (F2)

원리

- 안쪽에 **H-브리지**(스위치 4개)가 있어 모터 양 끝의 + − 를 바꾼다
- **DIR** 은 방향, **PWM** 은 켜진 비율 = 세기
- ESP32 는 신호만, 큰 전류는 배터리 → 드라이버 → 모터로 직접
- 모터가 멈출 때 생기는 역기전력을 드라이버 소자가 받아낸다

설치

- 신호 GND 를 **반드시** ESP32 GND 와 연결
- 전원선은 굵게(18AWG↑), 단자대는 단단히 조임
- 보드 위 **시험 버튼**으로 모터만 먼저 돌려 보기
- 방열 공간 확보 · 금속 바닥과 닿지 않게 절연

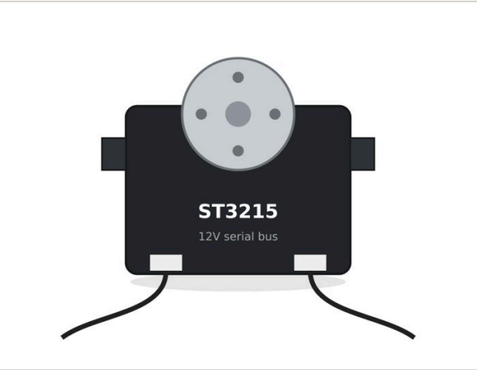
고장 시

- 한쪽만 안 돎 → 시험 버튼으로 모터 vs 신호 구분
- 둘 다 안 돎 → F2 퓨즈 · 전원 LED
- 제멋대로 움찔 → GND 미연결 · PWM 선 빠짐
- 뜨거움 → 모터 막힘·바퀴 걸림, 전류 확인

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 장치 ③ 조준

버스 서보 ST3215 — 선 하나에 서보 여러 개*



항목	값
전원	12V 계열 (판매처 사양 확인)
위치	0 ~ 4095 (12비트, 한 바퀴)
통신	반이중 직렬 1Mbps
ID	출하 시 1 → 하나는 2 로

원리

- 서보 안에 **모터 + 기어 + 위치센서 + 제어칩**이 다 들어 있다
- 목표 위치를 보내면 서보 혼자 그 각도로 가서 버틴다
- 선 하나로 보내고 받기를 **번갈아** 함 → 어댑터가 방향 전환
- ID 가 주소 — 같은 선 위에 ID 가 겹치면 둘 다 대답해서 깨짐

설치

- 서보를 **하나씩** 연결해 ID 바꾸기 (`X 1 2`)
- 어댑터 ↔ ESP32 TX·RX 교차, 어댑터 **모드 점퍼** 확인
- 조립 전 중심값(2048)으로 보내 놓고 혼(horn) 조립
- 케이블이 팬 회전에 끼지 않게 여유 루프

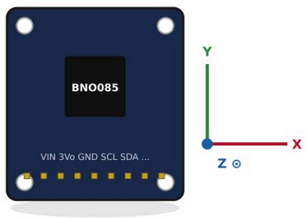
고장 시

- 무반응 → 12V(F3) · TX/RX · ID 중복
- `ok` 인데 안 움직임 → ok 는 '접수', 전원·점퍼
- 떨림 → 전원 전압 강하, 짐벌 무게 중심
- 방향 반대 → `SERVO_x_SIGN`

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 센서 ① 자세

IMU BNO085 — 기울기와 회전을 안다*



항목	값
축	가속도 3 + 자이로 3 + 지자기 3
출력	칩 안에서 자세 계산(퓨전)
연결	I2C 0x4A
전원	3.3V

원리

- **가속도**: 중력 방향 → 기울기 (흔들림에 약함)
- **자이로**: 회전 속도 → 빠른 회전 (오래 쓰면 오차 누적)
- **지자기**: 방위 (모터·쇠에 약함)
- 셋을 칩 안에서 섞어(퓨전) 서로의 약점을 메운다

설치

- 로봇 **회전 중심 가까이**, 바닥과 수평으로
- 축 방향(X 앞, Z 위)을 차체에 맞춰 붙이고 사진 기록
- 모터·배터리·스피커 자석에서 최대한 멀리
- 고무 받침·양면폼으로 진동 줄이기

고장 시

- `imu=MISSING` → SDA·SCL 바뀜 · 주소 · 3.3V
- 가만히 있는데 z≈9.8 아님 → 축 방향 · 보정
- 방위가 돌아감 → 자석 가까움 (지자기 빼고 사용)
- 가끔 끊김 → I2C 선 길이 · 듀폰 접촉

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 표시 ① 화면

OLED SSD1306 — 로봇의 작은 상태판*



항목	값
해상도	128 × 64
연결	I2C 0x3C (일부 0x3D)
전원	3.3V
역할	ID · HP · 탄약 · 배터리

원리

- 점 하나하나가 **스스로 빛나는** 화면 → 뒤 조명 없음
- ESP32 가 화면 그림을 칩 메모리에 보내면 칩이 계속 표시
- I2C 버스를 IMU 와 함께 씀 (주소로 구분)

설치

- 유리판이 얇다 → 나사는 **살짝만** 조이기
- 핀 순서(GND·VCC·SCL·SDA)는 모듈마다 다름 → 인쇄 확인
- 앞쪽에서 잘 보이는 곳, 레이저·서보 움직임과 안 부딪히게

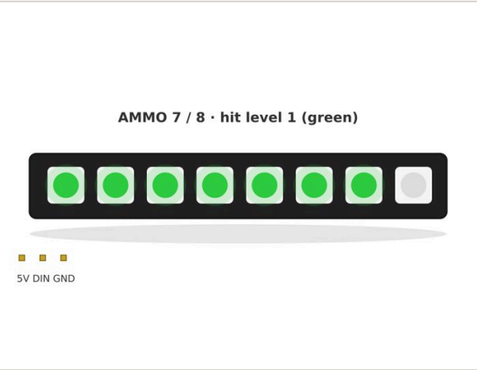
고장 시

- 안 켜짐 → 주소(0x3C/0x3D) · 핀 순서 · 3.3V
- 글자 깨짐·화면 일부 → I2C 속도 낮춰 시험(100kHz)
- IMU 까지 사라짐 → OLED 가 버스를 막음, 빼고 시험
- 금 간 화면 → 교체 (수리 불가)

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 표시 ② 탄창 LED

WS2812 — 한 선으로 색을 줄 세운다*



항목	값
전원	5V
데이터	한 선 직렬, 정해진 타이밍
개수	8개 (탄창 8발)
부품	레벨변환 · 330Ω · 1000µF

원리

- LED 마다 작은 칩이 있어 **앞의 24비트만 먹고** 나머지를 다음 LED 로 넘긴다
- 0 과 1 을 **펄스 길이**로 구분 → 타이밍이 생명
- 5V 장치라 HIGH 로 인정받으려면 3.3V 로는 모자랄 수 있다

설치

- DIN 화살표 방향 확인 (DOUT 에 꽂으면 무반응)
- GPIO 4 → **74AHCT125** → 330Ω → DIN
- 5V–GND 사이 **1000µF** (켜는 순간 보호)
- 밝기는 낮게 — 흰색 최대는 LED 하나에 수십 mA

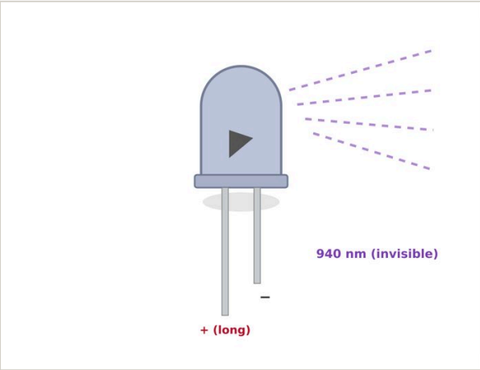
고장 시

- 하나도 안 켜짐 → DIN/DOUT 방향 · 5V · GND 공통
- 첫 LED 만 이상 → 첫 LED 손상 → 잘라내고 다음부터
- 색이 제멋대로 깜빡 → 레벨변환 없음 · 데이터선 길이
- 켤 때 ESP32 재부팅 → 밝기 과다 · 전원 용량

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 모의 레이저 ① 발사

적외선 LED TSAL6200 + MOSFET — 눈에 안 보이는 총알*



항목	값
파장	940nm (안 보임)
빛 퍼짐	반각 약 ±17°
스위치	AO3400A (게이트 GPIO 13)
신호	38kHz(예시) × 식별코드 프레임

원리

- ESP32 핀은 LED 에 큰 전류를 못 준다 → **MOSFET 이 대신** 켜고 끔
- 38kHz(예시) 로 깜빡이며 **로봇 ID** 를 실어 보냄 (리모컨 원리)
- 좁게 퍼질수록 멀리·정확하게, 넓으면 쉽게 맞지만 반사도 많음
- 직렬 저항 = 전류 제한 (옴의 법칙으로 계산)

설치

- 긴 다리 = + (애노드) → 전원 쪽
- 게이트 100Ω 직렬 + 100kΩ 풀다운 (부팅 중 오발사 방지)
- 카메라 광축과 **평행**하게 → 조준 = 사격
- 검은 튜브로 감싸 빔을 좁히면 오인 피격 감소

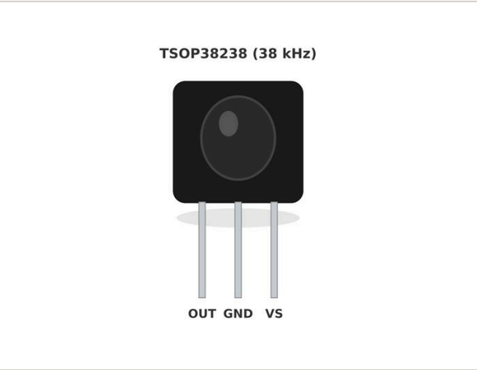
고장 시

- 안 쏨 → **휴대폰 카메라**로 LED 를 보면 보랏빛 깜빡임 확인
- 약함 → 저항값 · 전원 5V · MOSFET 방향
- 계속 켜짐 → 게이트 풀다운 없음 · MOSFET 단락
- LED 뜨거움 → 저항 너무 작음 · 듀티 과다

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 모의 레이저 ② 수신

적외선 수신기 TSOP38238 — 38kHz(예시) 만 골라 듣는다*



항목	값
반송파	38kHz(예시)
출력	신호 오면 LOW (평소 HIGH)
전원	3.3V (데이터시트 범위 내)
교육용 로봇	좌·우 2개 (GPIO 14 · 5)

원리

- 안에 **필터**가 있어 38kHz(예시) 로 깜빡이는 빛만 통과 → 형광등·햇빛 무시
- **자동 이득(AGC)**: 계속 켜진 빛은 잡음으로 보고 무시 → 끊어서 보내야 함
- 출력은 반송파가 벗겨진 **코드 모양** 그대로
- 펌웨어가 2초 연속 수신을 1회 피격으로 판정

설치

- 핀 순서 **1 OUT · 2 GND · 3 VS** (볼록 면이 앞)
- VS 에 **100Ω + 4.7µF** (전원 잡음 차단)
- OUT → 1kΩ → GPIO (부팅 충돌 방지)
- 로봇 좌·우 바깥쪽, 자기 IR LED 빛이 바로 안 닿게

고장 시

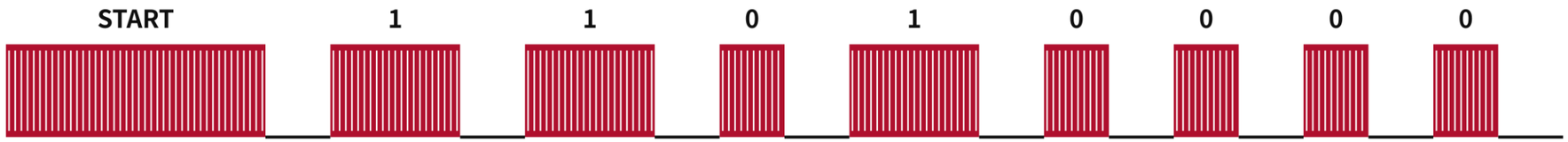
- 피격 안 됨 → 같은 ROBOT_ID? · 1m 정면부터 시험
- 항상 LOW → 핀 순서 · 전원 잡음
- 자기 총에 맞음 → 자기 ID 무시 확인 · 가림막
- 햇빛 아래 오작동 → 그늘·후드, 수신기 앞 가림 튜브

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 해설 · IR_CODEC.H

38kHz(예시) 를 끊어서 보내는 이유*

모의 레이저 식별코드 — 38kHz 를 끊어서 보낸다 (ID 11 예시)



빨간 칸 = 38kHz 로 깜빡이는 빛(mark). START 2400µs → 비트 8개(낮은 비트 먼저): 1 = 1200µs, 0 = 600µs, 사이 600µs

11 = 2진수 00001011 → 보내는 순서 1,1,0,1,0,0,0,0. 한 프레임 ≈ 16ms, 45ms 마다 반복 (5초 발사 동안 약 110번)

받는 쪽 (TSOP38238)

38kHz 빛만 골라 LOW 신호로 바꿔 줌
햇빛·형광등·리모컨 오인 ↓ (주파수 + 코드 이중 확인)
펄스 폭 해독 → 공격자 ID
같은 ID 가 150ms 이상 끊기지 않고 2초 → 유효 피격
자기 ID(벽 반사)는 무시 · 아군 ID 는 인정

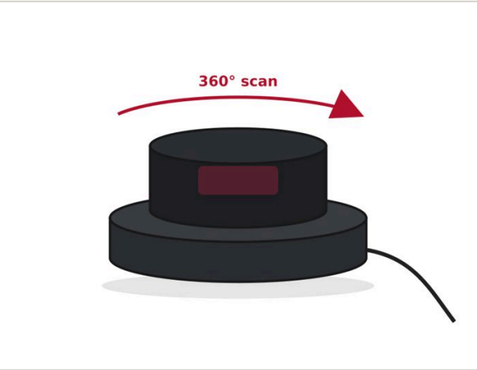
왜 그냥 켜 두면 안 되나?

TSOP 는 '계속 켜진 38kHz' 를 잡음으로 보고
스스로 감도를 낮춘다(AGC) → 수신 실패
그래서 짧게 끊어(burst) 보낸다
적외선은 눈에 안 보인다 → 발사 표시용 빨간 LED 를 따로 켬
레이저가 아니므로 눈 손상 위험이 낮다 (그래도 눈에 대고 쏘지 않기)

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

센서 ② 거리

LiDAR D500 — 빛으로 360° 거리를 잰다



항목	값
방식	DTOF (판매처 표기)
범위	360° 회전
연결	USB 어댑터 → Jetson
토픽	/scan

원리

- 레이저 펄스를 쏘고 **돌아오는 시간**으로 거리 계산
- 머리가 빙글빙글 돌며 각도마다 거리 한 점씩 → 주변 지도
- SLAM·장애물 회피의 기본 재료
- **검은색·유리·거울**은 빛이 안 돌아오거나 튕겨서 잘못 잼

설치

- 로봇 **가장 높은 곳**, 가리는 것 없이 (팬 틸트·안테나 주의)
- 바닥과 **수평** — 기울면 가까운 바닥을 벽으로 착각
- 전방 0° 방향을 차체 앞과 맞추고 표시
- USB 케이블 고정 (회전부에 닿지 않게)

고장 시

- 안 돎 → USB 전원 · 어댑터 LED
- 돌지만 점 없음 → 포트 이름 · 드라이버 호환(판매처 위키)
- 한쪽이 텅 빔 → 몸체가 가림 · 각도 범위 필터
- 사양·드라이버 호환은 확실하지 않음 → 판매처 문서로 확인

센서 ③ 깊이

OAK-D Lite — 두 눈으로 거리를 본다



항목	값
깊이	스테레오 흑백 카메라 2개
컬러	RGB 카메라 1개
처리	보드 안 VPU
연결	USB-C → USB3

원리

- 두 카메라가 **같은 점을 보는 위치 차이**(시차)로 거리 계산 — 사람 두 눈과 같음
- 두 카메라 사이 거리(베이스라인)가 길수록 먼 곳도 잘 잼
- 무늬 없는 흰 벽은 짝을 못 찾아 깊이가 비어 보임
- 보드 안 VPU 가 계산을 대신해 Jetson 부담 감소

설치

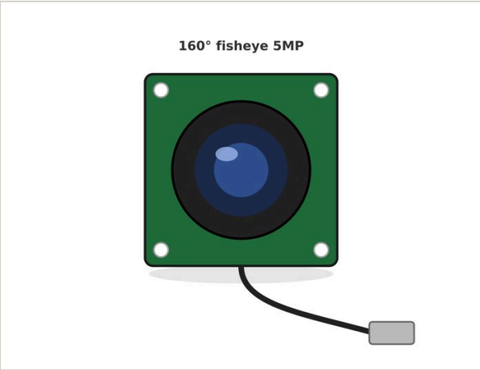
- **USB3 포트**(파란색)에 짧고 좋은 케이블로
- 앞을 보도록 단단히 고정 — 흔들리면 깊이도 흔들림
- 전력을 많이 씀 → **필요할 때만** 켜기 (대회 자료 권장)
- 렌즈 지문·먼지 닦기 (안경 천)

고장 시

- 인식 안 됨 → USB3 포트·케이블 바꾸기
- 중간에 끊김·재연결 → 전력 부족·케이블 불량
- 깊이가 구멍투성이 → 조명·무늬 부족 · 너무 가까움
- Jetson 느려짐 → 해상도·FPS 낮추기

센서 ④ 영상

USB 카메라 OV5640 — 팬틸트 위의 눈



항목	값
방식	UVC (드라이버 없이)
교육용 로봇	640 × 480 MJPG
토픽	/image_raw
이름	/dev/wingbot_cam

원리

- 렌즈 → 이미지 센서 → 카메라 칩이 압축(MJPG)해서 USB 로
- UVC 표준이라 리눅스에 꽂으면 바로 `/dev/video*`
- 해상도·FPS 를 올리면 USB 대역폭과 Jetson 계산이 늘어남
- 어두우면 노출 시간이 늘어 → 흔들림·느려짐

설치

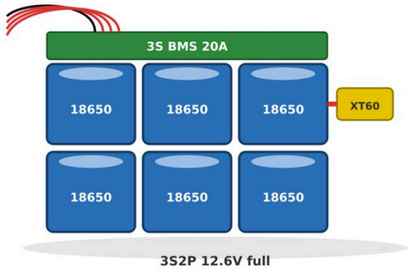
- 팬틸트 위 **레이저와 평행**하게 고정
- 항상 **같은 USB 포트** (udev 가 포트 위치로 이름 붙임)
- 케이블에 팬·틸트 회전 여유 루프
- 자동 초점 모델은 초점 고정 설정 확인

고장 시

- 안 잡힘 → `v4l2-ctl --list-devices`
- FPS 가 낮음 → YUYV 대신 MJPG · 조명
- 화면이 뒤집힘 → 카메라 설정 또는 노드에서 회전
- 가끔 멈춤 → USB 허브·전력, 포트 직결

교육용 전원 ①

배터리 3S2P + BMS + 전압 측정*



항목	값
구성	18650 셀 3직렬 × 2병렬
전압	만충 12.6V · 교육용 로봇 0% 기준 9.9V
보호	BMS + 메인 퓨즈 F1
측정	분압 100k/22k → GPIO 39

원리

- 셀 3개를 **직렬**로 → 전압 3배, 2줄 **병렬**로 → 용량 2배
- BMS 가 과충전·과방전·합선 때 **전기를 끊어** 셀을 보호
- ESP32 는 3.3V 까지만 → 저항 2개로 전압을 약 0.18배로 줄여 읽음
- 부하가 크면 전압이 순간 떨어짐 → 쉴 때 값으로 잔량 판단

설치

- 전용 충전기로만, 사람 있을 때만, 안전 백 안에서
- XT60 등 **극성 고정 커넥터** + 메인 스위치 + F1
- 고정은 단단히 — 날카로운 모서리·나사가 팩에 닿지 않게
- 분압점 전압이 3.3V 를 넘지 않는지 **배선 전에** 계산·측정

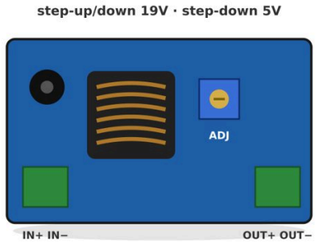
고장 시

- 전원 안 켜짐 → BMS 차단(과방전) → 충전기로 깨우기
- 배터리 % 이상 → 분압 저항값 · 멀티미터 값과 비교 보정
- 부풀음·뜨거움·냄새 → **즉시 사용 중지**, 다음 장 참고
- 주행 중 재부팅 → 전압 강하 · 커넥터 접촉 · 셀 노화

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 전원 ②

DC-DC 컨버터 — 12V 를 19V · 5V 로 바꾼다*



항목	값
19V 모듈	승압 → Jetson
5V 모듈	강압 → LED·IR·5V 장치
보호	갈래마다 퓨즈
조정	가변저항(트리머)

원리

- 스위치를 아주 빨리 켜고 끄며 코일·콘덴서로 전압을 바꾼다
- **강압(Buck)**: 높은 전압 → 낮게, **승압(Boost)**: 낮은 전압 → 높게
- 출력에 잔물결(리플)이 섞임 → 콘덴서로 줄임
- 출력 전류에는 한계 — 넘으면 전압이 무너짐

설치

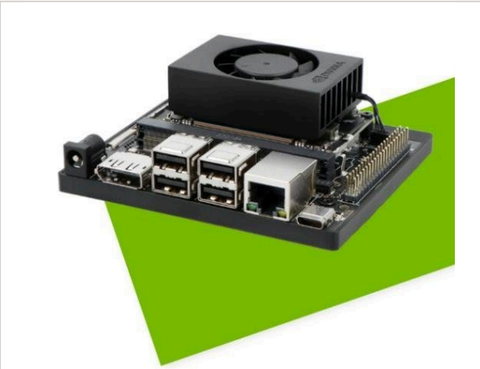
- **부하 없이** 먼저 켜서 출력 전압을 맞춘다 (19.0V · 5.0V)
- 맞춘 뒤 트리머에 매니큐어·핫글루로 고정
- 입력·출력 극성, IN/OUT 방향 확인 후 연결
- 방열 — 다른 부품과 띄우고 공기 흐르게

고장 시

- 출력 0V → 입력 퓨즈 · 극성 · 모듈 보호 동작
- 부하 걸면 전압이 뚝 → 용량 부족 (정격 확인)
- Jetson 이 가끔 꺼짐 → 19V 모듈 용량 · 배선 굵기
- 모듈 뜨거움 → 부하 과다, 더 큰 모듈로

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

Jetson Orin Nano 개발자킷 — 전원과 연결 포인트



항목	값
역할	영상·AI·ROS 2·서버 통신
전원	19V (DC-DC 19V 모듈)
USB	ESP32 · LiDAR · 카메라 · OAK-D
GPIO	3.3V, ADC 없음

원리

- GPU 가 있는 작은 컴퓨터 — 영상 인식처럼 계산이 큰 일을 맡음
- 리눅스라서 µs 타이밍은 약함 → 모터·서보·IR 은 ESP32 에 맡김
- 부하에 따라 전력이 크게 변함 → 전원 여유가 중요

설치

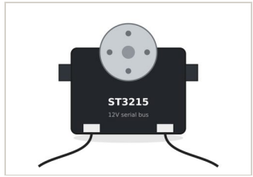
- 전원은 **맨 마지막**에 (배선 순서 8단계)
- 방열판·팬 막지 않기 — 위쪽 공기 흐름 확보
- USB 장치는 **항상 같은 포트**에 (udev 이름 고정)
- microSD/NVMe 고정, 케이블은 당겨지지 않게

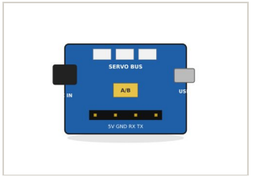
고장 시

- 갑자기 꺼짐 → 19V 전압·용량, F4 퓨즈
- USB 장치가 사라짐 → 전력 부족, 전원 USB 허브
- 느려짐·뜨거움 → 팬 동작 · 전력 모드(nvpmodel)
- 부팅 안 됨 → 저장장치 · 전원 LED · 모니터로 메시지 확인

교육용 해설 · STS_SERVO.H

패킷 한 줄 읽기 · 서보 ID 바꾸기*





FF FF 01 0A 03 29 32 00 08 00 00 8E 03 A3

바이트	FF FF	01	0A	03	29	32	00 08	00 00	8E 03	A3
뜻	시작	ID 1	길이 10	쓰기	주소 41	가속 50	위치 2048	시간 0	속도 910	체크섬

단위 변환

pan 100 × 0.5° = 50° → 50 × 4096/360 ≈ **569 step**

tilt 100 × 0.15° = 15° → **171 step**

2바이트 값은 **낮은 바이트 먼저** (2048 = 0x0800 → 00 08)

ID 바꾸기 (처음 한 번)

서보는 출하 때 모두 ID 1. **서보 1개만** 연결 → Serial Monitor에 `X 1 2` → 서보 전원 껐다 켜기 → 두 번째 서보 연결

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

PART 6

설치 확인과 보정

부품별 시험 → USB 이름 고정 → 센서 확인 → 실측 보정.

'된다'를 숫자로 확인하고, 숫자로 맞춘다.



교육용 부품별 시험

Serial Monitor 로 하나씩 확인 (연습 모드, 바퀴 띄움)*



시험	입력	기대 결과
통신	`P`	`P ok`
모터 방향	`M 300 300`	1초 동안 두 바퀴 앞으로 · `encL`, `encR` 둘 다 + 로 증가
속도 제어	`V 150 0` (0.5초 안에 반복)	천천히 전진, 멈추면 0.5초 뒤 자동 정지
회전 방향	`V 0 1000`	위에서 볼 때 **왼쪽(반시계)** 회전
팬틸트	`G 1 100 0 30 20`	카메라가 **오른쪽**으로 50° · `A 1 G ok 0`
발사	`F 2` → 다시 `F 3`	`fired 10000` → 빨간 LED 5초 · `reloading …`
OLED	`O 0 HELLO`	0번 줄 HELLO, 10초 뒤 기본화면
경기 잠금	`S mission standby` → `F 4`	`locked`, OLED [M-STBY]
피격 (로봇 2대)	A 로봇 `F 1`, B 로봇 수신기 정면 1m	2초 뒤 B: 부저, `H <A의 ID> L 3`, LED 초록
램프·부저	`K 1` / `K 0` / `B`	램프 켜짐/꺼짐 · 삐

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 USB 이름 고정

꽂는 순서가 바뀌어도 같은 이름 — udev 규칙*

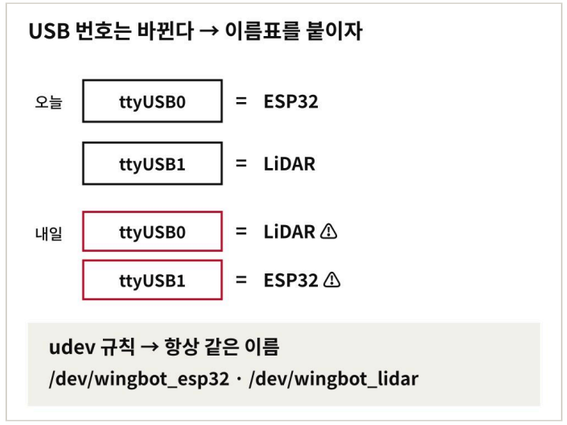
문제: ESP32 와 LiDAR 어댑터가 둘 다 `/dev/ttyUSB0·1` 로 잡히고, 재부팅마다 번호가 바뀔 수 있다.

해결: 장치 고유값으로 고정 이름을 만든다.

1. `udevadm info -a -n /dev/ttyUSB0 | grep -E "idVendor|idProduct|serial" | head`
2. `jetson/99-wingbot.rules` 의 `…_HERE` 를 내 값으로
3. `sudo cp 99-wingbot.rules /etc/udev/rules.d/`
4. `sudo udevadm control --reload && sudo udevadm trigger`
5. `ls -l /dev/wingbot_*` → esp32 · lidar · cam 3개

고정 이름	장치	구분 기준
/dev/wingbot_esp32	ESP32-DevKitC	USB 시리얼 번호
/dev/wingbot_lidar	D500 USB 어댑터	USB 시리얼 번호
/dev/wingbot_cam	팬틸트 카메라	꽂은 USB 포트 위치

> **팁**
> 두 장치의 USB 칩이 같아서(예: 둘 다 CP2102) 시리얼 번호까지 같다면, 카메라처럼 **포트 위치(ID_PATH)**로 구분하고 항상 같은 포트에 꽂으세요.



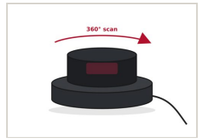
ttyUSB 번호 대신 고정 이름

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

센서

LiDAR · OAK-D Lite · 카메라 — 따로 켜서 확인





센서	확인 명령	정상
카메라	`v4l2-ctl --list-devices` → `v4l2-ctl -d /dev/wingbot_cam --list-formats-ext`	640×480 MJPG 지원
카메라 노드	`ros2 run wingbot_edu camera` → `ros2 topic hz /image_raw`	약 10Hz
LiDAR	launch 실행 후 `ros2 topic hz /scan` · RViz2 Fixed Frame = base_laser	회전 주기마다 점 구름
OAK-D Lite	`ros2 launch wingbot_edu wingbot_edu.launch.py oak:=true` (depthai-ros 설치 필요)	깊이 토픽 발행
IMU	`ros2 topic echo /imu/data --once`	가만히 있으면 가속도 z ≈ 9.8

확실하지 않은 부분

Waveshare D500 이 LDROBOT `ldlidar_stl_ros2` 드라이버(LD19 설정)와 호환되는지는 판매처 위키로 확인해야 합니다. 안 되면 판매처 제공 ROS 2 드라이버를 쓰고 토픽 이름만 `/scan` 으로 맞추세요.

전력

OAK-D Lite 는 USB3 로 전기를 많이 씁니다. 대회 자료도 'Depth 카메라는 필요할 때만'이라고 권장합니다. 기본값을 꺼 둔 이유입니다.

교육용 해설 · CONFIG.H

꼭 실측해서 고칠 4가지*

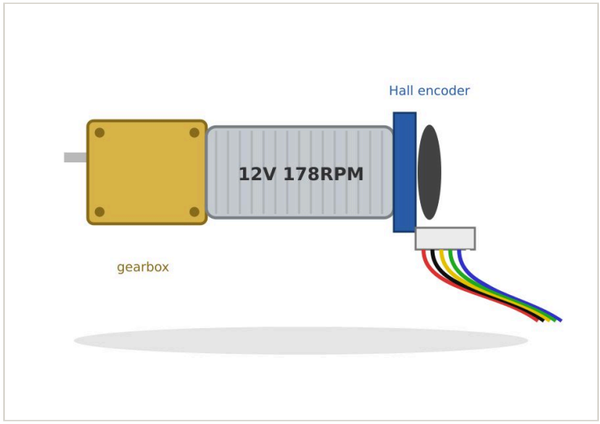
항목	실측 방법	틀리면
`ENC_COUNTS_PER_REV`	바퀴에 테이프 표시 → `M 300 300` 대신 손으로 10바퀴 돌리고 encL/10	속도·거리가 비율만큼 틀림
`WHEEL_DIAMETER_M`	1m 직진 후 오도메트리 x 와 줄자 비교 → 지름 × (실제/측정)	직진 거리 오차
`TRACK_WIDTH_M`	제자리 10바퀴 회전 → yaw 합계와 비교해 비율 보정	회전 각도 오차 (지도가 휘어짐)
`MOTOR_L/R_INVERT`	`M 300 300` → 두 바퀴 모두 앞으로 가는지	제자리에서 뱅글뱅글

ROBOT_ID
로봇마다 다른 번호(1~255). 같으면 서로 맞혀도 '자기 반사'로 무시됩니다.

경기 규칙 숫자
`FIRE_ON_MS 5000` · `FIRE_PERIOD_MS 10000` · `HIT_HOLD_MS 2000` · `HIT_LOCK_MS 10000` — 대회 규칙과 같은 값



바퀴 속도 차이 = 회전



엔코더 모터

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 보정

서보 중심 · IMU · 배터리 전압 — 실측으로 맞추기*

대상	왜	방법	고칠 곳
팬 · 틸트 중심	조립 각도마다 정면 위치가 다름	`G 1 0 0 0 0` → 카메라가 정면을 볼 때까지 중심값 조정	`SERVO_CENTER_PAN/TILT`
팬 · 틸트 방향	대회 규칙: pan + = 오른쪽, tilt + = 위	`G 1 50 0 0 0` → 오른쪽으로 가는지	`SERVO_x_SIGN`
IMU 축	차체와 센서 축이 어긋나면 회전이 틀림	가만히: 가속도 z ≈ +9.8 / 왼쪽으로 돌릴 때 자이로 z 가 +	설치 방향 또는 노드에서 축 변환
IMU 보정	자이로·지자기 초기 오차	켠 뒤 몇 초간 가만히 · 8자 돌리기(지자기 사용 시)	BNO085 보정 상태 확인
배터리 전압	저항 오차로 % 가 틀림	멀티미터 버스 전압 vs `T` 의 bat 값 비교	`BAT_DIV_RATIO` 를 실측 비율로
IR 사거리	LED 저항·빔 각도에 따라 다름	정면 1m → 2m → 3m · 좌우 15°·30° 표 작성	저항값 · 빔 튜브 길이

팁

보정값은 로봇마다 다릅니다. config.h 맨 위에 **로봇 번호 · 날짜 · 측정값**을 주석으로 남겨 두세요.

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 비교

같은 기능, 다른 방식 — ESP32 와 Jetson*





	ESP32 (Arduino C++)	Jetson (Python)
아날로그 입력	칩 안에 ADC → `analogReadMilliVolts`	**없음** → ADS1115 를 I2C 로
핀 번호	GPIO 번호 하나	BOARD(물리) / BCM / I2C 버스 번호(7?) 를 구분
바이트 순서	—	ADS1115 는 큰 바이트 먼저 (STS 서보와 반대)
타이밍	µs 단위 정확, 전원 켜면 바로 실행	OS 가 끼어들어 ms 단위 흔들림, 부팅 수십 초
설치	IDE 에서 Upload	apt 패키지 + 실행 권한
잘하는 일	모터·서보·IR 처럼 **정확한 시간**	영상 AI·지도·경로처럼 **큰 계산**
시험	같은 분압(2.2V) → 12.2V	같은 분압(2.2V) → 12.2V (자동 시험 일치)

> **결론**
> 그래서 WINGBOT 도, 교육용 로봇도 **둘 다** 씁니다. 정확한 시간은 ESP32, 큰 계산은 Jetson.

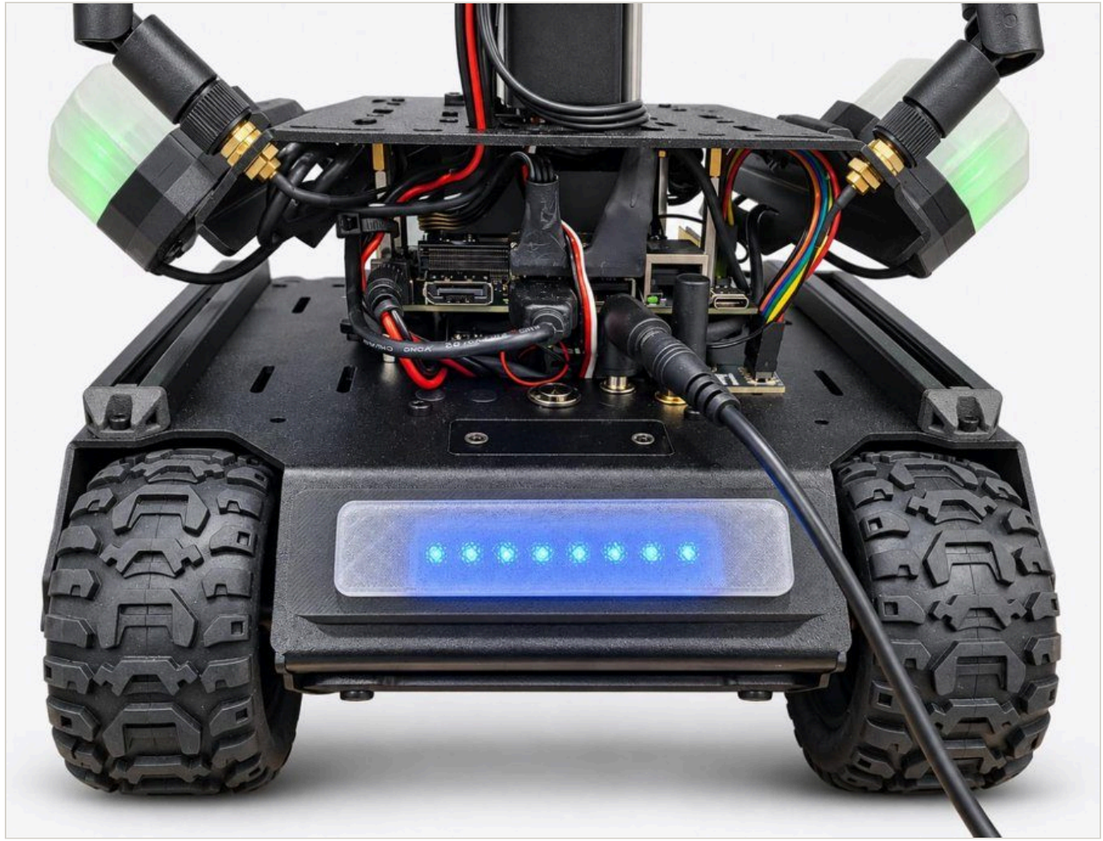
* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

PART 7

고장 대응

진단 순서 · 멀티미터 · 증상표 · 부품 교체 · 배터리 사고 · 경기 당일 점검.

먼저 끄고, 전원 → 연결 → 신호 → 소프트웨어 순서로.



진단 순서

고장은 아래에서 위로 — 5단계

1. **보기·냄새·온도** — 끄고 눈으로: 탄 자국, 빠진 선, 뜨거운 부품, 녹은 피복
2. **전원** — 배터리 전압 → 퓨즈 → 12V 버스 → 19V · 5V · 3.3V 순서로 측정
3. **GND·연결** — 공통 GND 통전, 커넥터 흔들어 보기, 냉납
4. **신호** — Serial Monitor 로 부품 하나씩 (`P · M · G · F · O`)
5. **소프트웨어** — 설정값(INVERT · ID · 포트 이름) → 그다음 코드

반으로 나누기

어디가 문제인지 모르면 **가운데**를 먼저 잽니다. 예) 모터가 안 돎 → MDD10A 시험 버튼: 돌면 앞쪽(ESP32·신호), 안 돌면 뒤쪽(전원·모터).

하나만 바꾸기

한 번에 하나만 바꾸고 시험합니다. 두 개를 같이 바꾸면 무엇이 고친 건지 모릅니다.

기록

증상 · 바꾼 것 · 결과를 한 줄씩. 다음 고장 때 시간이 반으로 줄어듭니다.

멀티미터

멀티미터로 재는 4가지 — 켜고 재기 / 끄고 재기

측정	전원	프로브 위치	다이얼	예
전압	**켠 상태**	재는 곳과 GND 에 **나란히(병렬)**	V⎓ (직류)	12V 버스, 3V3 핀 3.2~3.4V
통전	**끈 상태**	선의 양 끝	삐 표시	GND 끼리 '삐', 이웃 핀끼리는 안 삐
저항	**끈 상태**, 가능하면 회로에서 뺌	부품 양 끝	Ω	풀업 10kΩ, 분압 100k/22k
전류	켠 상태	회로를 끊고 **사이에 (직렬)**	A, **빨간 선을 A 단자로**	대기 전류, 모터 전류

다이오드 모드

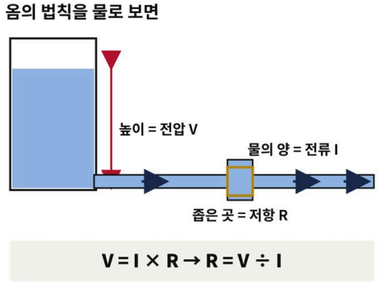
LED 극성 · MOSFET 바디 다이오드 확인에 씁니다. 빨간 프로브가 +.

위험

전류 모드인 채로 전압을 재면 배터리를 합선시키는 것과 같습니다 → 멀티미터 퓨즈가 끊기거나 불꽃. 재고 나면 빨간 선을 **V 단자로** 돌려놓기.

자주 하는 실수

- 전원 켠 채 저항 측정 → 엉뚱한 값·계측기 손상
- 직류(V⎓)가 아니라 교류(V~)에 놓음
- 검은 선을 GND 가 아닌 곳에 댐
- 프로브가 미끄러져 이웃 핀과 합선 → 끝이 가는 프로브 사용



전압 = 높이 차 → 두 점 사이를 잰다

교육용 **증상표 ①**

전원 · 부팅 — 켜지지 않거나 자꾸 꺼질 때*

증상	먼저 의심	확인 방법	조치
아무것도 안 켜짐	메인 스위치 · F1 · BMS 차단	배터리 단자 전압 → 퓨즈 앞뒤 전압	퓨즈 교체 전 **합선 원인부터** 찾기
퓨즈가 또 끊어짐	합선 · 모터 막힘 · 역극성	전원 끄고 각 갈래 +/− 사이 통전	갈래 하나씩 연결하며 원인 찾기
ESP32 만 안 켜짐	USB 케이블(충전 전용) · 3.3V 단락	3V3–GND 저항, 다른 케이블	데이터 케이블로 교체
모터 켤 때 ESP32 재부팅	전압 강하(브라운아웃) · GND 공유 잘못	모터 켤 때 3.3V·5V 떨어짐 측정	GND 한 점, 전원 분리, 콘덴서
Jetson 이 갑자기 꺼짐	19V 모듈 용량 · F4 · 커넥터	부하 중 19V 측정	정격 큰 모듈, 굵은 선
타는 냄새 · 연기	역극성 · 합선 · 과전류	**즉시 메인 스위치 OFF**, 배터리 분리	식은 뒤 탄 부품·주변 배선 점검
부품이 손대기 어렵게 뜨거움	과전류 · 잘못된 전압	끄고 식힌 뒤 전압·전류 측정	원인 고치기 전에는 다시 켜지 않기

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 **증상표 ②**

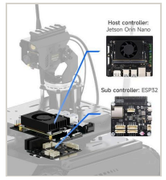
구동 · 조준 · 센서 — 움직임이 이상할 때*

증상	먼저 의심	확인 방법	조치
바퀴 하나만 안 돎	모터 선 · 병렬 연결 · 기어	그 모터만 직접 12V(짧게) 연결	모터 교체 또는 배선 수리
한쪽 3바퀴 전부 안 돎	드라이버 채널 · PWM/DIR 선	MDD10A 시험 버튼	신호선·GND 확인
직진이 휘어짐	좌우 모터 차이 · 엔코더 · 타이어	`V 150 0` 에서 encL/encR 비교	PI 게인 · 바퀴 지름 실측
엔코더 값이 튐	풀업 없음 · 모터선과 같이 묶임	모터 끄고 손으로 돌려 비교	10kΩ 풀업, 선 분리·꼬기
팬틸트 떨림·멈춤	12V 강하 · 무게 중심 · ID	움직일 때 서보 전원 측정	전원 굵기, 짐벌 균형
피격이 안 됨 / 너무 쉬움	ID · 거리 · 빔 각도 · 반사	1m 정면 → 거리·각도 표	빔 튜브 · 수신기 가림
IMU·OLED 둘 다 사라짐	I2C 버스 막힘	하나씩 빼고 부팅	고장 모듈 교체, 선 짧게
LiDAR·카메라가 가끔 끊김	USB 전력 · 케이블 · 진동	`dmesg -w` 로 재연결 로그	짧은 케이블, 고정, 전원 허브

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 오류 점검

증상 → 먼저 볼 곳*



증상	먼저 의심	확인
업로드 'Connecting…' 에서 멈춤	자동 부팅모드 진입 실패	BOOT 버튼 누른 채 업로드
Serial Monitor 깨진 글자	baud 불일치	115200
모터 한쪽만 반대로	모터 극성	`MOTOR_x_INVERT` 변경 (배선 말고)
모터가 혼자 움찔	GND 미연결 · 떠 있는 핀	공통 GND · PWM 핀 풀다운
엔코더 값이 튐 / 0	풀업 없음 · 전원 5V	34·35 에 10kΩ, 엔코더 VCC 3.3V
팬틸트 무반응	서보 ID 둘 다 1 · TX/RX 바뀜 · 12V 없음	`X 1 2` · 교차 연결 · F3 퓨즈
G 응답은 ok 인데 안 움직임	ok = 접수일 뿐	어댑터 모드 점퍼 · 전원
피격이 안 됨	자기 ID · 거리 · 각도	ROBOT_ID 다르게 · 1m 정면부터
피격이 너무 쉽게 됨	반사 · 넓은 빛	IR LED 에 검은 튜브(빔 좁히기)
/odom 안 나옴	포트 이름 · Serial Monitor 가 점유	`ls /dev/wingbot_*`
로봇이 0.5초마다 멈칫	cmd_vel 이 10Hz 미만	`ros2 topic hz /cmd_vel`
Jetson 이 갑자기 꺼짐	전압 강하 · 19V 컨버터 용량	F4 · 컨버터 정격 5A 이상

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 부품 교체

부품 교체 순서와 예비품 상자*

1. 메인 스위치 OFF → **배터리 분리** → 콘덴서 방전 위해 잠시 대기
2. 교체할 부품의 결선 **사진** 찍기 · 선에 이름표
3. 부품 교체 (같은 모델·같은 방향·같은 핀 순서 확인)
4. **전원 없이** 통전·합선 확인
5. 배선 순서표의 해당 단계부터 다시 측정하며 전원 연결
6. Serial Monitor 부품별 시험 → 보정값 다시 확인
7. 고장 기록: 날짜 · 증상 · 원인 · 교체 부품

예비품	수량	이유
퓨즈 (각 정격)	갈래마다 2~3	가장 자주 소모
ESP32-DevKitC (펌웨어 미리 올림)	1	핀 손상 시 즉시 교체
TSOP38238 · TSAL6200 · AO3400A	각 2	작고 싸고 잘 망가짐
JGB37-520 모터	1	기어 파손 대비
ST3215 서보 (ID 설정해 둠)	1	충돌로 기어 손상
DC-DC 5V · 19V 모듈 (출력 맞춰 둠)	각 1	Jetson 보호
USB 데이터 케이블 · 점퍼선 · 열수축	넉넉히	접촉 불량 대응

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

배터리 사고

배터리가 부풀거나, 뜨겁거나, 연기가 날 때

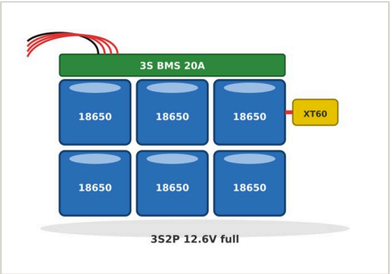
징후	할 일
팩이 부풀었다	**사용·충전 중지**. 누르거나 구멍 내지 않기. 불연 용기에 따로 보관 후 지정 폐기
손대기 어렵게 뜨겁다	메인 스위치 OFF · 충전기 분리 · 주변 가연물 치우기 · 식을 때까지 관찰
단 냄새 · 쉿 소리 · 연기	**사람 먼저 대피**, 환기, 담당 교사·안전관리자에게 알림, 필요하면 119
떨어뜨림 · 찍힘	겉이 멀쩡해도 한동안 따로 두고 온도·모양 관찰
커넥터 녹음 · 피복 탐	과전류·합선 흔적 → 원인 찾기 전 재사용 금지

예방

전용 충전기 · 사람이 있을 때만 충전 · 안전백 · 과방전 금지(전압 낮으면 바로 충전) · 장기 보관은 약 50% 충전 · 금속 공구와 따로 보관

확실하지 않은 부분

리튬이온 화재 진압 방법은 셀 크기·장소에 따라 권고가 다릅니다. **학교·실습실의 안전 규정과 소화 설비 안내**를 먼저 확인하고 수업 전에 공유하세요.



3S2P 팩 + BMS

교육용 경기 당일

출발 전 하드웨어 점검표 — 3분이면 끝난다*

#	점검	기준
1	배터리 전압 (멀티미터)	12.0V 이상, 충전 시각 기록
2	퓨즈 · 메인 스위치 · XT60	체결·흔들림 없음
3	커넥터 흔들어 보기 (신호선 전부)	OLED·LED 가 꺼지지 않음
4	OLED 기본 화면	ID · HP 4/4 · AMMO 8 · BAT %
5	바퀴 띄우고 `V 150 0`	좌우 같은 방향, encL·encR 증가
6	팬틸트 중심 · 오른쪽 · 위	부호 규칙 일치
7	IR 발사 (휴대폰 카메라)	보랏빛 깜빡임 5초
8	LiDAR 회전 · 카메라 영상	`/scan` · `/image_raw` 주기 정상
9	나사·케이블타이·렌즈	풀림 없음, 렌즈 깨끗

팀 역할

한 사람이 표를 읽고, 한 사람이 확인하고, 체크 표시. **읽는 사람과 하는 사람을 나누면** 빠뜨림이 줄어듭니다.

경기 사이

배터리 교체·충전 → 1·4번 다시 · 부딪힌 경기 뒤에는 6·8번 다시



출발 전 3분 점검

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

교육용 **놓치기 쉬운 팁**

한 장 요약*



- **단순 동작 ≠ 안전한 설계** — 켜진다고 끝이 아니라 전력·퓨즈·발열까지
- **USB 충전 전용 케이블**은 포트가 안 보인다 → 데이터 케이블
- Arduino IDE **Board = ESP32 Dev Module, Port** 다시 확인
- Serial Monitor **baud 115200**, 줄 끝 Newline
- ESP32 · Jetson GPIO 는 **3.3V** — 5V 직접 금지
- Jetson **BOARD(물리) 번호 ≠ BCM 번호**
- 모터 전원은 따로, **GND 는 공통**
- L298N 이면 **ENA 점퍼**, 발열, 2V 강하 · 모터 **Stall 전류 × 병렬 수**
- MOSFET 는 VGS(th) 말고 **실제 게이트 전압에서 RDS(on)**
- **바디 다이오드 ≠ 플라이백 다이오드**
- 게이트 저항 = 돌입·울림 억제, **게이트 풀다운** = 부팅 중 오동작 방지
- pan + 오른쪽 · angular.z + 왼쪽 · **ok 는 접수**

* **교육용**(WB-EDU) 제안 — 대회용 공식 사양이 아닙니다. **대회용** 로봇의 실제 구동은 ROS 2 API 로 합니다 (부록 D).

재고, 기록하고, 하나씩 바꾼다.

하드웨어는 원리를 알면 무섭지 않다. 전원은 단계마다 재고, 고장은 아래에서 위로 찾고, 바꾼 것은 기록한다.

다음 단계: 펌웨어 코드 · ROS 2 노드 · 경기 서버 연동 (본 강의)

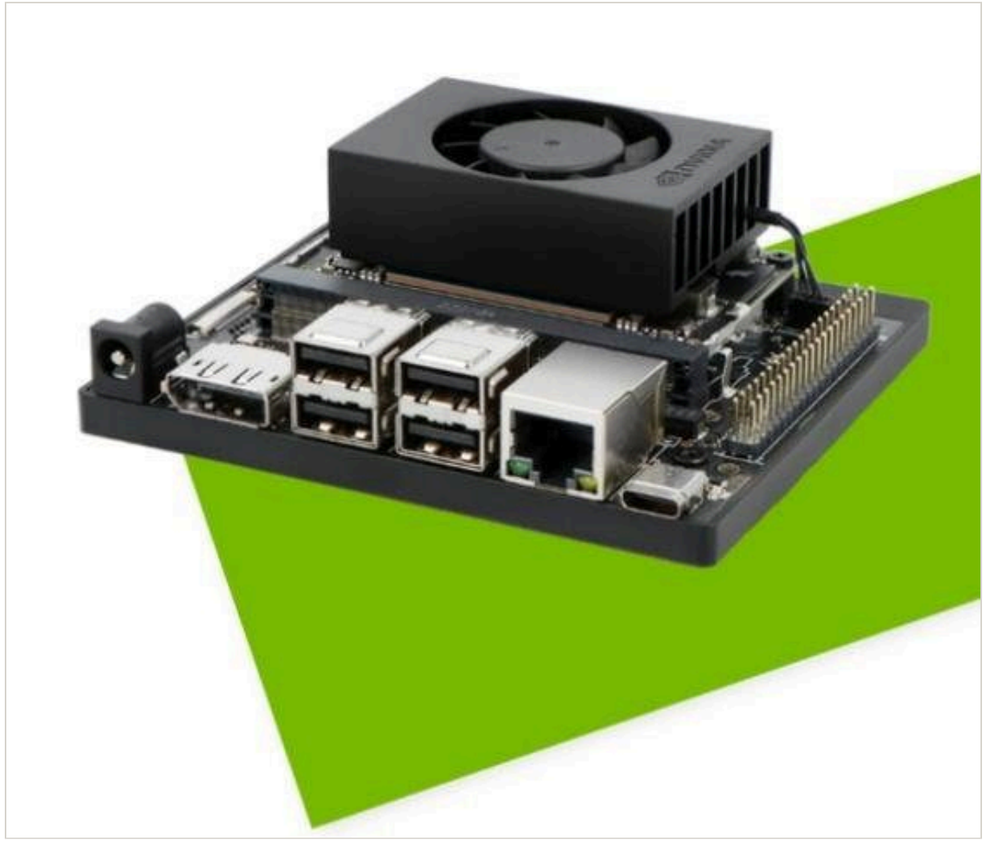


부록

대회용 API · 구동 예시 · 사양 비교

대회용 로봇은 공개된 ROS 2 API 와 경기 서버 HTTP API 로 움직인다. 기능마다 사양과 활용 예시.

** 표시 장은 원리, 이 부록은 실전.*



대회용 부록 D · 대회용 API 와 구동 예시

대회 로봇 API 전체 지도 (ROS 2)

기능	Topic / Service	방식	핵심 값 (2차 사전학습 자료)
상태	`/mil/state`	구독	hp · hp_max · battery_pct · 2Hz
경기 단계	`/mil/match`	구독	mode(practice / mission) · cmd(idle·standby·start·pause·resume·report·stop)
주행	`/cmd_vel`	발행	linear.x ±0.6 m/s · angular.z ±3 rad/s (+ 좌회전) · 10Hz 반복
위치	`/odom`	구독	position x·y (m) · orientation 쿼터니언 · 20Hz
팬틸트	`/mil/gimbal`	서비스	pan · tilt −100~100 (pan + 오른쪽 0.5°/단위, tilt + 위 0.15°/단위) · speed 1~270 · acc 1~254
레이저	`/mil/fire`	서비스	1회 5초 발광 · 발사 시작 10초 후 재발사 → fired / empty / locked / reloading · left_ms
카메라	`/image_raw`	구독	640×480 BGR · 최대 30FPS (10FPS 이하 권장) · 팬틸트 카메라
IMU	`/imu/data`	구독	자세(쿼터니언) · 각속도 · 가속도 · 20Hz
OLED	`/mil/oled`	발행	line 0~3 · 한 줄 21자 · 0.5초 이상 간격 · 10초 뒤 기본 화면

> **팀 프로그램**
> `~/team/team_node.py` · standby 때 자동 실행 · 비정상 종료 시 경기당 3회 재실행 · `step()` 0.1초 이내 · AI 추론은 별도 스레드

대회용 부록 D · 대회용 API 와 구동 예시

경기 단계와 상태 — /mil/match · /mil/state

cmd	뜻	제어
idle	경기 없음	잠금
standby	시작 대기 (팀 노드 자동 실행)	잠금
start	경기 진행	**허용**
pause	일시정지 (레이저 꺼짐 · 모터 정지)	잠금
resume	재개 — 명령을 다시 보내야 움직임	**허용**
report	종료 후 보고 (M6)	잠금
stop	경기 종료 → 팀 노드 종료	잠금

활용

① 매 주기 control 을 먼저 확인하고 False 면 정지 명령만 ② resume 뒤 주행·팬틸트 명령 재전송 ③ hp·battery_pct 로 교전 회피 판단

```
# 제어 가능 여부 (자료의 판단식)
def on_match(m):
    self.control = (m.mode == 'practice'
                    or m.cmd in ('start', 'resume'))
    self.phase = m.cmd

def on_state(s):
    self.hp, self.bat = s.hp, s.battery_pct

# QoS: depth=1 · TRANSIENT_LOCAL
#  → 늦게 구독해도 마지막 상태를 바로 받는다
# 연습(practice) 모드: 단계 잠금 없음,
#   탄약·HP 자동 회복 (피격 잠금·발사 간격은 적용)
# 경기(mission) 모드: 운영 시스템이 전환,
#   탄약·HP 회복 없음
```

주행 — /cmd_vel · /odom

필드	뜻	범위
linear.x	전진(+) · 후진(−)	−0.6 ~ 0.6 m/s
angular.z	좌회전(+) · 우회전(−)	−3 ~ 3 rad/s
position.x / y	출발점 기준 위치	m
orientation	방향 (쿼터니언 → yaw)	x, y, z, w

규칙

약 **10Hz 로 반복** 전송 — 0.5초 끊기면 자동 정지 · 여러 노드가 동시에 `/cmd_vel` 보내지 않기 · `/odom` 은 qos_profile_sensor_data

활용 예

직진 1m: odom 거리 1.0 될 때까지 linear.x 0.2 · 90° 회전: yaw 차이가 90° 될 때까지 angular.z 0.8 · 거점 진입: 마지막 30cm 는 0.1 m/s

```
from geometry_msgs.msg import Twist
pub = node.create_publisher(Twist, '/cmd_vel', 10)

msg = Twist()
msg.linear.x = 0.15          # 전진 0.15 m/s
msg.angular.z = 0.0
for _ in range(10):          # 10Hz × 1초
    pub.publish(msg)
    time.sleep(0.1)
pub.publish(Twist())         # 정지 (속도 0)

# /odom 에서 방향(yaw) 구하기
q = odom.pose.pose.orientation
yaw = math.degrees(math.atan2(
    2*(q.w*q.z + q.x*q.y),
    1 - 2*(q.y*q.y + q.z*q.z)))
```

팬틸트 — /mil/gimbal

필드	뜻	범위
pan	좌우 · + 오른쪽 · 1단위 0.5°	−100 ~ 100
tilt	상하 · + 위 · 1단위 0.15°	−100 ~ 100
speed	이동 속도 (°/s)	1 ~ 270
acc	가속도 (권장 20)	1 ~ 254

응답	뜻
ok	명령 **접수** (도착이 아님)
locked	제어 잠금 (대기·일시정지·피격 후 10초)
응답 없음	실패 · 시간 초과

활용 예

화면 중심과 표적 중심의 가로 오차(px) × 계수 → pan 보정 · ok 뒤 0.3~0.5초 기다렸다가 발사 · 탐색 때 pan 을 −60 → 0 → 60 으로 훑기

```
req = Gimbal.Request()
req.pan, req.tilt = 0.0, 0.0      # 정면
req.speed, req.acc = 80.0, 20.0
future = client.call_async(req)
rclpy.spin_until_future_complete(
    node, future, timeout_sec=3.0)
# → ok / locked

# 표적을 화면 가운데로 (비례 조준 예)
err_x = cx - 320                  # 640×480 의 중심
pan  += 0.05 * err_x              # + 오른쪽
pan   = max(-100, min(100, pan))

# Gimbal 타입의 패키지 이름은 로봇에서 확인
#   ros2 service list -t | grep gimbal
```

레이저 발사 — /mil/fire

message	뜻	할 일
fired	발사 성공 (5초 발광 시작)	2초 이상 표적에 유지
reloading	재장전 대기	left_ms 만큼 기다림
locked	잠금 (대기·일시정지·보고·종료 · 피격 후 10초)	위치 이동에 시간 쓰기
empty	탄약 없음	교전 중지 · M3/M5 에 집중

규칙

탄창 8발 · 1회 5초 발광 · 발사 시작 10초 후 재발사 · 유효타격 = 수광센서 2초 연속 · 경기 모드는 탄약 회복 없음 · HP 0 이어도 탄약이 있고 제한이 없으면 발사 가능

레이저 안전

사람 눈에 직접 닿으면 시력 손상 위험 — 발사 전 주변 확인, 눈 방향 발사 금지

```
future = client.call_async(Fire.Request())   # 파라미터 없음
rclpy.spin_until_future_complete(
    node, future, timeout_sec=3.0)
r = future.result()

if r.message == 'fired':
    hold_aim(seconds=5)           # 발광 5초 동안 조준 유지
elif r.message == 'reloading':
    wait_ms = r.left_ms           # 남은 대기 시간
elif r.message == 'locked':
    move_to_cover()               # 피격 후 10초 등
elif r.message == 'empty':
    self.can_fire = False

# 판정은 표적·상대 로봇의 수광센서가 자동으로 한다
# (참가팀이 보고하는 API 없음)
```

카메라 · IMU · OLED — /image_raw · /imu/data · /mil/oled

API	핵심	활용 예
`/image_raw`	640×480 · BGR(OpenCV) · 최대 30FPS, 10FPS 이하 권장 · 팬틸트 카메라 (Depth 카메라 아님)	표적·객체·상대 UGV 인식, 안테나 색 피아식별
`/imu/data`	roll · pitch · yaw · 각속도 · 가속도 · 20Hz	기울기·충격 감지 시 즉시 정지, 회전 각도 확인
`/mil/oled`	OledLine(line 0~3, text 21자) · 0.5초 이상 간격	팀 상태·탐색 단계 표시로 현장 디버깅

BGR 주의

색 순서가 파랑·초록·빨강입니다. RGB 로 학습한 모델에는 `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)` 로 바꿔 넣으세요.

```
from cv_bridge import CvBridge
from rclpy.qos import qos_profile_sensor_data

def on_image(msg):
    frame = CvBridge().imgmsg_to_cv2(msg, 'bgr8')
    # frame.shape == (480, 640, 3)
    self.latest = frame        # 추론은 별도 스레드에서

node.create_subscription(Image, '/image_raw',
                         on_image, qos_profile_sensor_data)

# 충격 감지 → 정지
g = math.sqrt(a.x**2 + a.y**2 + a.z**2) / 9.80665
if g > 2.0:
    cmd_pub.publish(Twist())

oled.publish(OledLine(line=0, text='TEAM ALPHA'))
time.sleep(0.5)                # 간격 유지
```

구동 조합 예시 — 표적을 찾아 멈추고 쏘기

```
def step(self):                        # 0.1초 이내로 끝낼 것
    if not self.control:               # 잠금이면 정지만
        self.cmd.publish(Twist()); return

    box = self.detector.latest()       # 별도 스레드의 추론 결과
    if box is None:                    # 1) 탐색: 천천히 회전
        self.drive(0.0, 0.6); return

    err = box.cx - 320                 # 2) 조준: 화면 중심 오차
    if abs(err) > 20:
        self.drive(0.0, 0.0)           #    멈추고
        self.aim(self.pan + 0.05*err) #    팬으로 맞춘다
        return

    if self.is_ally(box):              # 3) 피아식별 (아군 오사도 피격)
        return
    r = self.fire()                    # 4) 발사
    if r.message == 'fired':
        self.hold_until = now() + 5.0 #    5초 동안 조준 유지
    elif r.message == 'locked':
        self.retreat()                 #    피격 후 10초 → 엄폐
```

단계	쓰는 API
잠금 확인	`/mil/match`
탐색 회전	`/cmd_vel` (angular.z)
영상 인식	`/image_raw`
조준	`/mil/gimbal`
발사	`/mil/fire`
상태 표시	`/mil/oled`
생존 판단	`/mil/state` (hp)

이 코드는 예시입니다

함수 이름(drive · aim · fire · is_ally)과 계수 0.05 · 오차 20px 은 설명용입니다. 실제 값은 연습 모드에서 맞추세요.

대회용 부록 D · 대회용 API 와 구동 예시

경기 서버 API (HTTP) — 임무 받기와 보고

단계	메서드 · 경로	보내는 값	받는 값
상태 확인	GET `/me`	mission_code	phase · ready
준비	POST `/ready`	mission_code	accepted
1차 임무 (0:00)	GET `/order`	mission_code · wait=120	order.tactical_command
드론 화면 (선택)	GET `/drone`	mission_code	최신 JPEG 1장 (1초마다 갱신)
M1 해석	POST `/interpretation`	target · search_sectors · base	accepted · received
M3 객체	POST `/detection`	sectors (type · count)	accepted · received
2차 임무 (4:00)	GET `/target`	mission_code · wait=120	order.objective
M6 위험도	POST `/situation`	ranking (상대 호출부호 순)	accepted · received

공통

주소 `http://<서버IP>:8000/api/report/…` · 팀 구분은 **mission_code**(영문+숫자 8자리) — GET 은 쿼리, POST 는 JSON 본문 · 서버가 먼저 보내지 않음 → 팀이 요청

시간

phase: waiting → ready → running → reporting → finished · 서버는 남은 시간을 주지 않음 → 팀이 직접 잼 · 마감은 **서버가 받은 시각** · 단계가 바뀐 뒤 보고는 **409**

API 가 없는 것

M2 · M4 는 수광센서가, M5 는 거점 진입 센서가 자동 판정합니다.

확실하지 않음

필드의 자료형 · wait 단위와 최댓값 · 409 외 오류 응답은 웹 문서에 없습니다 → 리허설 서버 응답의 `received` 로 확인

경기 서버 활용 예시 — 받고, 해석하고, 제때 보고

```
import os, requests
BASE = os.environ["MAICON_BASE"]          # http://<서버IP>:8000
CODE = os.environ["MAICON_MISSION_CODE"]  # 코드에 직접 쓰지 않는다
API  = BASE + "/api/report"

def get(path, wait=None):
    p = {"mission_code": CODE}
    if wait: p["wait"] = wait
    return requests.get(API + path, params=p,
                        timeout=(wait or 0) + 10)

def post(path, body):
    r = requests.post(API + path, timeout=5,
                      json={"mission_code": CODE, **body})
    if r.status_code == 409:              # 단계가 이미 바뀜
        log("too late: " + path)
    return r

post("/ready", {})
cmd = get("/order", wait=120).json()["order"]["tactical_command"]
post("/interpretation", {"target": t, "search_sectors": s, "base": b})
# ... 4:00 ...
goal = get("/target", wait=120).json()["order"]["objective"]
# ... report 신호 후 30초 안 ...
post("/situation", {"ranking": ["R-UGV-2", "R-UGV-3", "R-UGV-1"]})
```

시각	할 일
준비	`/ready` → `/order` 대기
0:00	전술명령 수신 → M1 해석 → `/interpretation`
경기 중	`/drone` 으로 상대 위치 기록 · `/detection`
3:00~	`/target` 대기 시작
4:00	목표거점 수신 → 출발
report 신호	30초 안에 `/situation`

주의

ranking 의 호출부호 값과 sectors 의 구조는 예시입니다. 실제 값은 대회 규약 문서와 리허설 응답으로 확인하세요.

대회용 부록 E · 업체 사양과 대회 문서 사양 비교

업체 공개 사양 ↔ 대회 문서 사양 — 다른 곳 확인

항목	업체 공개 사양 (유사 제품)	대회 문서 (2차 사전학습 자료)	차이 · 확인할 것
구동 방식	바퀴 6개 · **구동륜 4개** (6륜 4WD) · 엔코더 모터 4개	**6WD** Wheel · 6WD 고안정성 구동 플랫폼 · 메탈기어모터 (개수 표기 없음)	**표기가 다름** → 실물에서 모터 개수 확인
최대 속도	기본 최대 1.3 m/s	API linear.x ±0.6 m/s	대회용은 API 에서 제한
배터리	18650 × 3 (포함 안 됨) · 3S UPS 모듈	12.6V · **3S2P 6셀** · UPS 모듈 · 약 40분	대회용이 2배 용량 구성
메인 컴퓨터	Jetson Orin Nano (4GB 키트 선택)	Jetson Orin Nano **8GB** · M.2 SSD 256GB	메모리 다름
운영체제	데비안 북웜 (호스트 시스템 지원 표기)	Ubuntu 22.04 · ROS 2 Humble (Docker)	표기 다름
크기 · 무게	253 × 231 × 289 mm · 약 2.7 kg	250 × 230 × **320** mm · 약 2.9 kg	대회용이 더 높고 무거움
팬틸트	ST3215 서보 · 360° 전방향 표기	ST3215 · 수평 100° · 수직 30°	대회용은 범위 제한 (API ±100)
카메라	160° · 5MP	160도 · 5MP	같음
제어 방법	웹 애플리케이션 / Jupyter Lab	ROS 2 API (team_node.py) · Jupyter Lab :8888	웹 조종 앱 사용 가능 여부 미확인
레이저 · 수광센서 · 피아식별 안테나	표기 없음	레이저 발광/수광 · 안테나 색상 · LAMP · 스피커	대회용 추가 장치
타이어	지름 80 mm · 폭 42.5 mm · 제자리 회전	표기 없음	오도메트리 계산 시 실측

읽을 때 주의

대회 문서는 **제조사·제품명을 밝히지 않습니다.** 왼쪽은 사양과 모양이 비슷한 시판 제품(6륜 4WD · Jetson Orin · 팬틸트)의 공개 사양이며, 대회용 로봇이 이 제품과 같다고 확인된 것은 아닙니다. 차이가 나는 항목은 **대회 문서와 실물**을 기준으로 하고, 불확실하면 운영사무국에 문의하세요.